

```
0001 # =====
0002 # FILE: backend\config.py
0003 # =====
0004 """
0005 全局配置模块
0006 =====
0007 集中管理数据库连接、COR、Agent 模型 API Key 等环境配置。
0008 使用 pydantic-settings 从环境变量 /.env 文件读取。
0009 """
0010
0011 from functools import lru_cache
0012 from typing import List, Optional
0013
0014 from pydantic import Field
0015 from pydantic_settings import BaseSettings, SettingsConfigDict
0016
0017
0018 class Settings(BaseSettings):
0019     """应用全局配置"""
0020
0021     # ---- 基础 ----
0022     APP_NAME: str = "Research Colab Backend"
0023     APP_VERSION: str = "0.1.0"
0024     DEBUG: bool = True
0025
0026     # ---- CORS ----
0027     # 允许跨域的前端来源 (Vite dev server 默认 5173)
0028     CORS_ORIGINS: List[str] = [
0029         "http://localhost:5173",
0030         "http://127.0.0.1:5173",
0031     ]
0032
0033     # ---- PostgreSQL 数据库 (asyncpg URL) ----
0034     DATABASE_URL: str = Field(
0035         default="postgresql+asyncpg://postgres:postgres@localhost:5432/research_colab",
0036         description="SQLAlchemy 异步数据库连接串",
0037     )
0038
0039     # ---- WebSocket 协同 ----
0040     WS_MAX_CLIENTS_PER_DOC: int = 64
0041
0042     # ---- AI 模型接口 (预留: DeepSeek-V3 / 通义千问) ----
0043     # 使用 OpenAI 兼容协议, 可切换 base_url 实现不同厂商
0044     LLM_BASE_URL: str = "https://api.deepseek.com/v1" # DeepSeek-V3
0045     LLM_API_KEY: str = "" # 从环境变量注入
0046     LLM_MODEL: str = "deepseek-chat"
0047
0048     # 通义千问备用配置
0049     QWEN_BASE_URL: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
0050     QWEN_API_KEY: str = ""
```

```
0051 QWEN_MODEL: str = "qwen-plus"
0052
0053 # ---- 水印算法参数 (Kirchenbauer) ----
0054 WATERMARK_GAMMA: float = 0.5 # 绿名单比例
0055 WATERMARK_DELTA: float = 2.0 # logits 偏移强度
0056 WATERMARK_SECRET_KEY: bytes = b"research-colab-watermark-secret-key-2026"
0057
0058 # ---- 操作日志哈希链 ----
0059 HASHCHAIN_SALT: str = "research-colab-oplog-salt"
0060
0061 model_config = SettingsConfigDict(
0062     env_file=".env",
0063     env_file_encoding="utf-8",
0064     extra="ignore",
0065 )
0066
0067
0068 @lru_cache
0069 def get_settings() -> Settings:
0070     """获取全局单例配置 (lru_cache 避免重复加载 .env)"""
0071     return Settings()
0072
0073
0074 # 模块级导出, 便于直接 from config import settings
0075 settings: Settings = get_settings()
0076 # =====
0077 # FILE: backend/database.py
0078 # =====
0079 """
0080 异步数据库连接与会话管理
0081 =====
0082 使用 SQLAlchemy 2.0 async API + asyncpg 驱动连接 PostgreSQL。
0083
0084 **自动回退机制**:
0085 - 优先尝试连接 PostgreSQL (asyncpg)
0086 - 若连接失败 (服务未启动/无权限), 自动回退到本地 SQLite 文件数据库
0087   (backend/research_colab.db), 保证系统无需数据库环境也能直接启动运行
0088
0089 提供:
0090 - `engine`: 全局异步引擎 (自动选择 PostgreSQL/SQLite)
0091 - `async_session_factory`: 异步会话工厂
0092 - `get_db()`: FastAPI 依赖, 用于请求级会话
0093 - `init_db()`: 启动时建表 (生产环境建议改用 Alembic 迁移)
0094 """
0095
0096 import logging
0097 from typing import AsyncGenerator, Optional
0098
0099 from sqlalchemy.ext.asyncio import (
0100     AsyncSession,
```

```
0101     async_sessionmaker,
0102     create_async_engine,
0103 )
0104 from sqlalchemy.orm import DeclarativeBase
0105 from sqlalchemy import event
0106
0107 from config import settings
0108
0109 logger = logging.getLogger("unicorn.error")
0110
0111
0112 # -----
0113 # ORM 声明基类：所有 models.py 中的模型继承此类
0114 # -----
0115 class Base(DeclarativeBase):
0116     """SQLAlchemy 2.0 声明式基类"""
0117
0118     pass
0119
0120
0121 # -----
0122 # 数据库引擎创建 (PostgreSQL 优先, 失败回退 SQLite)
0123 # -----
0124 def _create_engine():
0125     """创建异步引擎。
0126
0127     尝试连接 PostgreSQL；若不可用，则回退到 SQLite 文件数据库。
0128     """
0129     pg_url = settings.DATABASE_URL
0130
0131     # 1. 尝试 PostgreSQL
0132     try:
0133         import asyncpg # noqa: F401
0134
0135         engine = create_async_engine(
0136             pg_url,
0137             echo=settings.DEBUG,
0138             pool_size=10,
0139             max_overflow=20,
0140             pool_pre_ping=True,
0141             connect_args={"timeout": 3}, # 3 秒连接超时，快速失败回退
0142         )
0143
0144         # 启动时做一次真实连接测试
0145         import asyncio
0146
0147         async def _probe() -> bool:
0148             try:
0149                 async with engine.connect() as conn:
0150                     await conn.execute(__import__("sqlalchemy").text("SELECT 1"))
```

```
0151     return True
0152 except Exception:
0153     return False
0154
0155     # 注意: 这里不能用 asyncio.run 如果事件循环已在运行。
0156     # 改为惰性检测: 在 init_db 时兜底
0157     try:
0158         loop = asyncio.get_running_loop()
0159     except RuntimeError:
0160         loop = None
0161
0162     if loop is None:
0163         # 模块导入时无事件循环, 直接探测
0164         ok = asyncio.run(_probe())
0165         if ok:
0166             logger.info("    数据库连接: PostgreSQL @ %s", pg_url.rsplit("@", 1)[-1])
0167             return engine
0168             logger.warning("⚠ PostgreSQL 不可用, 回退到 SQLite 文件数据库")
0169         else:
0170             # 事件循环已在运行 (如 uvicorn 导入), 直接假定 PostgreSQL,
0171             # 真正的兜底在 init_db() 中执行
0172             logger.info("    数据库引擎: PostgreSQL (事件循环内惰性验证)")
0173             return engine
0174
0175     except ImportError:
0176         logger.warning("⚠ asyncpg 未安装, 直接使用 SQLite 文件数据库")
0177     except Exception as exc: # 连接失败等
0178         logger.warning("⚠ PostgreSQL 连接失败 (%s), 回退到 SQLite", exc)
0179
0180     # 2. 回退: SQLite 文件数据库
0181     sqlite_path = "/research_colab.db"
0182     engine = create_async_engine(
0183         f"sqlite+aiosqlite:/// {sqlite_path}",
0184         echo=settings.DEBUG,
0185     )
0186
0187     # SQLite 需要手动开启外键约束
0188     @event.listens_for(engine.sync_engine, "connect")
0189     def _set_sqlite_pragma(dbapi_connection, connection_record): # noqa: ANN001
0190         cursor = dbapi_connection.cursor()
0191         cursor.execute("PRAGMA foreign_keys=ON")
0192         cursor.close()
0193
0194     logger.info("    数据库连接: SQLite @ %s", sqlite_path)
0195     return engine
0196
0197
0198 engine = _create_engine()
0199
0200 async_session_factory: AsyncSessionmaker[AsyncSession] = async_sessionmaker(
```

```
0201     bind=engine,
0202     class_=AsyncSession,
0203     expire_on_commit=False,
0204     autoflush=False,
0205 )
0206
0207
0208 # -----
0209 # FastAPI 依赖注入
0210 # -----
0211 async def get_db() -> AsyncGenerator[AsyncSession, None]:
0212     """
0213     请求级数据库会话依赖。
0214
0215     用法:
0216     @router.get("/documents")
0217     async def list_documents(db: AsyncSession = Depends(get_db)):
0218         ...
0219     """
0220     async with async_session_factory() as session:
0221         try:
0222             yield session
0223         finally:
0224             pass
0225
0226
0227 # -----
0228 # 初始化建表 (开发环境)
0229 # -----
0230 async def init_db() -> None:
0231     """
0232     根据 models.py 中定义的 ORM 模型创建所有表。
0233
0234     若 PostgreSQL 连接失败 (首次真正使用连接时才暴露), 在此处兜底重建 SQLite 引擎。
0235     注意: 生产环境应使用 Alembic 迁移工具而非此函数。
0236     """
0237     global engine, async_session_factory
0238
0239     # 延迟导入, 确保 models 已注册到 Base.metadata
0240     import models # noqa: F401 (触发模型注册)
0241
0242     # 尝试建表; 失败则回退到 SQLite
0243     try:
0244         async with engine.begin() as conn:
0245             await conn.run_sync(Base.metadata.create_all)
0246         logger.info(" 数据库表创建/校验完成")
0247     except Exception as exc:
0248         # PostgreSQL 不可用 -> 切到 SQLite
0249         if engine.dialect.name == "postgresql":
0250             logger.warning("⚠ PostgreSQL 建表失败 (%s), 回退到 SQLite 文件数据库", exc)
```

```
0251
0252     sqlite_path = "/research_colab.db"
0253     new_engine = create_async_engine(
0254         f"sqlite+aiosqlite:/// {sqlite_path}",
0255         echo=settings.DEBUG,
0256     )
0257
0258     @event.listens_for(new_engine.sync_engine, "connect")
0259     def _set_sqlite_pragma(dbapi_connection, connection_record): # noqa: ANN001
0260         cursor = dbapi_connection.cursor()
0261         cursor.execute("PRAGMA foreign_keys=ON")
0262         cursor.close()
0263
0264     engine = new_engine
0265     async_session_factory = async_sessionmaker(
0266         bind=engine,
0267         class_=AsyncSession,
0268         expire_on_commit=False,
0269         autoflush=False,
0270     )
0271
0272     async with engine.begin() as conn:
0273         await conn.run_sync(Base.metadata.create_all)
0274     logger.info("    SQLite 数据库表创建完成 @ %s", sqlite_path)
0275     else:
0276         raise
0277     # =====
0278     # FILE: backend\models.py
0279     # =====
0280     """
0281     SQLAlchemy ORM 模型定义
0282     =====
0283     对应数据库表:
0284     1. User - 用户表
0285     2. Document - 文档表 (含 yjs_state 二进制字段)
0286     3. OpLog - 操作日志表 (prev_hash/current_hash/operation_jsonb)
0287     4. WatermarkRecord - 水印记录表
0288     5. Literature - 文献资源表 (调研 Agent 检索语料)
0289     6. PermissionConfig - 权限配置表 (文档/用户访问控制)
0290
0291     技术要点:
0292     - 使用 SQLAlchemy 2.0 Mapped / mapped_column 类型注解风格
0293     - 操作日志采用"哈希链"结构实现版权溯源与防篡改
0294     - 类型跨方言设计: PostgreSQL 使用原生 UUID/JSONB/BYTEA,
0295       SQLite 自动降级为 CHAR(32)/JSON/BLOB (便于本地无数据库直接运行)
0296     """
0297
0298     import uuid
0299     from datetime import datetime
0300     from typing import List, Optional
```

```
0301
0302 from sqlalchemy import (
0303     String,
0304     Text,
0305     LargeBinary,
0306     Integer,
0307     SmallInteger,
0308     Float,
0309     ForeignKey,
0310     UniqueConstraint,
0311     Index,
0312     CheckConstraint,
0313     JSON,
0314     DateTime,
0315     TIMESTAMP,
0316     Uuid,
0317     func,
0318 )
0319 from sqlalchemy.dialects.postgresql import UUID as PG_UUID
0320 from sqlalchemy.dialects.postgresql import JSONB as PG_JSONB
0321 from sqlalchemy.dialects.postgresql import BYTEA as PG_BYTEA
0322 from sqlalchemy.orm import Mapped, mapped_column, relationship
0323
0324 from database import Base
0325
0326
0327 # -----
0328 # 跨方言类型定义
0329 # -----
0330 # PostgreSQL 使用原生 UUID, 其他方言 (SQLite) 使用 CHAR(32) 字符串
0331 UUID_TYPE = Uuid().with_variant(PG_UUID(as_uuid=True), "postgresql")
0332
0333 # PostgreSQL 使用原生 JSONB, 其他方言使用 JSON
0334 JSONB_TYPE = JSON().with_variant(PG_JSONB, "postgresql")
0335
0336 # PostgreSQL 使用原生 BYTEA, 其他方言使用 LargeBinary (BLOB)
0337 BYTEA_TYPE = LargeBinary().with_variant(PG_BYTEA, "postgresql")
0338
0339
0340 # 通用 GUID 主键生成函数
0341 def _gen_uuid() -> uuid.UUID:
0342     """生成 UUID 主键"""
0343     return uuid.uuid4()
0344
0345
0346 # =====
0347 # 1. 用户表
0348 # =====
0349 class User(Base):
0350     """
```

```

0351 用户表: 人类用户与 AI Agent 账号统一在该表中建模。
0352 AI Agent 通过 role='ai_agent' 区分。
0353 """
0354
0355 __tablename__ = "users"
0356 __table_args__ = (
0357     CheckConstraint(
0358         "role IN ('admin', 'researcher', 'collaborator', 'ai_agent')",
0359         name="ck_users_role",
0360     ),
0361     Index("idx_users_role", "role"),
0362 )
0363
0364 id: Mapped[uuid.UUID] = mapped_column(
0365     UUID_TYPE, primary_key=True, default=_gen_uuid
0366 )
0367 username: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)
0368 email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
0369 display_name: Mapped[str] = mapped_column(String(128), nullable=False)
0370 avatar_url: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
0371 role: Mapped[str] = mapped_column(
0372     String(32), nullable=False, default="researcher"
0373 )
0374 created_at: Mapped[datetime] = mapped_column(
0375     TIMESTAMP(timezone=True), nullable=False, server_default=func.now()
0376 )
0377 updated_at: Mapped[datetime] = mapped_column(
0378     TIMESTAMP(timezone=True),
0379     nullable=False,
0380     server_default=func.now(),
0381     onupdate=func.now(),
0382 )
0383
0384 # 反向关系
0385 documents: Mapped[List["Document"]] = relationship(
0386     back_populates="owner", foreign_keys="Document.owner_id"
0387 )
0388 op_logs: Mapped[List["OpLog"]] = relationship(back_populates="user")
0389
0390 def __repr__(self) -> str:
0391     return f"<User {self.username} ({self.role})>"
0392
0393
0394 # =====
0395 # 2. 文档表
0396 # =====
0397 class Document(Base):
0398     """
0399     文档表:
0400     - yjs_state: 存储 Yjs CRDT 文档的完整状态快照 (Update Message 二进制)

```



```
0401 - content: 明文快照, 供快速预览与全文检索 (与 yjs_state 最终一致)
0402 - watermark_status: 0-未检测 1-含AI水印 2-纯人类创作
0403 ""
0404
0405 __tablename__ = "documents"
0406 __table_args__ = (
0407     Index("idx_documents_owner", "owner_id"),
0408     Index("idx_documents_updated_at", "updated_at"),
0409 )
0410
0411 id: Mapped[uuid.UUID] = mapped_column(
0412     UUID_TYPE, primary_key=True, default=_gen_uuid
0413 )
0414 title: Mapped[str] = mapped_column(String(512), nullable=False)
0415 owner_id: Mapped[uuid.UUID] = mapped_column(
0416     UUID_TYPE,
0417     ForeignKey("users.id", ondelete="CASCADE"),
0418     nullable=False,
0419 )
0420 # Yjs 协同状态快照 (二进制)
0421 yjs_state: Mapped[Optional[bytes]] = mapped_column(BYTEA_TYPE, nullable=True)
0422 # 明文快照
0423 content: Mapped[str] = mapped_column(Text, nullable=False, default="")
0424 # 0-未检测 1-含AI水印 2-纯人类
0425 watermark_status: Mapped[int] = mapped_column(
0426     SmallInteger, nullable=False, default=0
0427 )
0428 agent_session_id: Mapped[Optional[uuid.UUID]] = mapped_column(
0429     UUID_TYPE, nullable=True
0430 )
0431 created_at: Mapped[datetime] = mapped_column(
0432     TIMESTAMP(timezone=True), nullable=False, server_default=func.now()
0433 )
0434 updated_at: Mapped[datetime] = mapped_column(
0435     TIMESTAMP(timezone=True),
0436     nullable=False,
0437     server_default=func.now(),
0438     onupdate=func.now(),
0439 )
0440
0441 # 关系
0442 owner: Mapped["User"] = relationship(
0443     back_populates="documents", foreign_keys=[owner_id]
0444 )
0445 collaborators: Mapped[List["User"]] = relationship(
0446     secondary="document_collaborators",
0447     lazy="selectin",
0448 )
0449 op_logs: Mapped[List["OpLog"]] = relationship(back_populates="document")
0450 watermarks: Mapped[List["WatermarkRecord"]] = relationship(
```

```
0451     back_populates="document"
0452 )
0453
0454 def __repr__(self) -> str:
0455     return f"<Document {self.title}>"
0456
0457
0458 # 文档-协作者中间表 (多对多)
0459 class DocumentCollaborator(Base):
0460     """文档-协作者关联表"""
0461
0462     __tablename__ = "document_collaborators"
0463     __table_args__ = (
0464         UniqueConstraint("document_id", "user_id", name="uq_doc_collab"),
0465     )
0466
0467     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
0468     document_id: Mapped[uuid.UUID] = mapped_column(
0469         UUID_TYPE,
0470         ForeignKey("documents.id", ondelete="CASCADE"),
0471         nullable=False,
0472     )
0473     user_id: Mapped[uuid.UUID] = mapped_column(
0474         UUID_TYPE,
0475         ForeignKey("users.id", ondelete="CASCADE"),
0476         nullable=False,
0477     )
0478
0479
0480 # =====
0481 # 3. 操作日志表 (版权溯源哈希链)
0482 # =====
0483 class OpLog(Base):
0484     """
0485     操作日志表 (哈希链结构):
0486     - prev_hash: 前一条操作日志的 current_hash ("" 表示链首)
0487     - current_hash: sha256(prev_hash + operation_json + created_at)
0488     - operation: 操作内容 JSON (PostgreSQL: JSONB / SQLite: JSON)
0489
0490     每条日志可通过 current_hash 串联成链。一旦某条历史日志被修改,
0491     其后所有 current_hash 均失效, 服务端可快速校验链完整性。
0492     """
0493
0494     __tablename__ = "op_logs"
0495     __table_args__ = (
0496         CheckConstraint(
0497             "op_type IN ('insert', 'delete', 'replace', 'ai_generate', 'watermark_checked')",
0498             name="ck_oplogs_type",
0499         ),
0500         Index("idx_oplogs_doc_created", "doc_id", "created_at"),
```

```

0501     Index("idx_oplogs_prev_hash", "prev_hash"),
0502     Index("idx_oplogs_op_type", "op_type"),
0503 )
0504
0505     id: Mapped[uuid.UUID] = mapped_column(
0506         UUID_TYPE, primary_key=True, default=_gen_uuid
0507     )
0508     doc_id: Mapped[uuid.UUID] = mapped_column(
0509         UUID_TYPE,
0510         ForeignKey("documents.id", ondelete="CASCADE"),
0511         nullable=False,
0512     )
0513     user_id: Mapped[uuid.UUID] = mapped_column(
0514         UUID_TYPE,
0515         ForeignKey("users.id", ondelete="CASCADE"),
0516         nullable=False,
0517     )
0518     op_type: Mapped[str] = mapped_column(String(32), nullable=False)
0519     # 操作内容 JSONB
0520     operation: Mapped[dict] = mapped_column(JSONB_TYPE, nullable=False, default=dict)
0521     # 哈希链字段
0522     prev_hash: Mapped[str] = mapped_column(String(64), nullable=False, default="")
0523     current_hash: Mapped[str] = mapped_column(
0524         String(64), nullable=False, unique=True
0525     )
0526     created_at: Mapped[datetime] = mapped_column(
0527         TIMESTAMP(timezone=True), nullable=False, server_default=func.now()
0528     )
0529
0530     # 关系
0531     document: Mapped["Document"] = relationship(back_populates="op_logs")
0532     user: Mapped["User"] = relationship(back_populates="op_logs")
0533
0534     def __repr__(self) -> str:
0535         return f"<OpLog {self.op_type} @ {self.doc_id}>"
0536
0537
0538     # =====
0539     # 4. 水印记录表
0540     # =====
0541     class WatermarkRecord(Base):
0542         """
0543         水印记录表:
0544         记录某文档中 AI 生成内容的水印参数 (Kirchenbauer 方案)。
0545         用于后续检测时还原绿/红名单, 验证文本是否包含水印。
0546         """
0547
0548         __tablename__ = "watermark_records"
0549         __table_args__ = (
0550             Index("idx_watermarks_doc", "doc_id"),

```

```
0551 )
0552
0553 id: Mapped[uuid.UUID] = mapped_column(
0554     UUID_TYPE, primary_key=True, default=_gen_uuid
0555 )
0556 doc_id: Mapped[uuid.UUID] = mapped_column(
0557     UUID_TYPE,
0558     ForeignKey("documents.id", ondelete="CASCADE"),
0559     nullable=False,
0560 )
0561 model_name: Mapped[str] = mapped_column(String(128), nullable=False)
0562 # 绿名单比例
0563 gamma: Mapped[float] = mapped_column(Float, nullable=False, default=0.5)
0564 # logits 偏移强度
0565 delta: Mapped[float] = mapped_column(Float, nullable=False, default=2.0)
0566 # 哈希密钥 (用于还原绿名单)
0567 secret_key: Mapped[bytes] = mapped_column(BYTEA_TYPE, nullable=False)
0568 # 涉及水印的 token 序列 (调试用)
0569 token_seq: Mapped[dict] = mapped_column(JSONB_TYPE, nullable=False, default=list)
0570 created_at: Mapped[datetime] = mapped_column(
0571     TIMESTAMP(timezone=True), nullable=False, server_default=func.now()
0572 )
0573
0574 # 关系
0575 document: Mapped["Document"] = relationship(back_populates="watermarks")
0576
0577 def __repr__(self) -> str:
0578     return f"<WatermarkRecord {self.model_name} gamma={self.gamma} delta={self.delta}>"
0579
0580
0581 # =====
0582 # 5. 文献资源表
0583 # =====
0584 class Literature(Base):
0585     """
0586     文献资源表: 调研 (Research) Agent 的检索语料库。
0587
0588     支持按标题/摘要/关键词检索, 前端展示检索结果并可一键插入
0589     引文到协作文档。
0590     """
0591
0592     __tablename__ = "literature"
0593     __table_args__ = (
0594         Index("idx_literature_keywords", "keywords"),
0595         Index("idx_literature_year", "year"),
0596     )
0597
0598     id: Mapped[uuid.UUID] = mapped_column(
0599         UUID_TYPE, primary_key=True, default=_gen_uuid
0600     )
```

```
0601 title: Mapped[str] = mapped_column(String(512), nullable=False)
0602 authors: Mapped[str] = mapped_column(String(512), nullable=False, default='')
0603 year: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
0604 source: Mapped[str] = mapped_column(String(128), nullable=False, default='')
0605 abstract: Mapped[str] = mapped_column(Text, nullable=False, default='')
0606 keywords: Mapped[str] = mapped_column(String(256), nullable=False, default='')
0607 url: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
0608 created_at: Mapped[datetime] = mapped_column(
0609     TIMESTAMP(timezone=True), nullable=False, server_default=func.now()
0610 )
0611
0612 def __repr__(self) -> str:
0613     return f"<Literature {self.year} {self.title[:40]}>"
0614
0615
0616 # =====
0617 # 6. 权限配置表
0618 # =====
0619 class PermissionConfig(Base):
0620     """
0621     权限配置表: 文档级访问控制策略。
0622
0623     每个文档可配置:
0624     - 协作模式 (collab_mode): open(公开协作) / invited(受邀协作)
0625     - 水印策略 (watermark_policy): enforce(强制水印) / optional(可选)
0626     - 导出限制 (export_policy): allow / deny
0627     """
0628
0629     __tablename__ = "permission_configs"
0630     __table_args__ = (
0631         UniqueConstraint("doc_id", name="uq_perm_doc"),
0632         CheckConstraint(
0633             "collab_mode IN ('open', 'invited')", name="ck_perm_collab_mode"
0634         ),
0635         CheckConstraint(
0636             "watermark_policy IN ('enforce', 'optional')",
0637             name="ck_perm_watermark_policy",
0638         ),
0639         CheckConstraint(
0640             "export_policy IN ('allow', 'deny')", name="ck_perm_export_policy"
0641         ),
0642     )
0643
0644     id: Mapped[uuid.UUID] = mapped_column(
0645         UUID_TYPE, primary_key=True, default=_gen_uuid
0646     )
0647     doc_id: Mapped[uuid.UUID] = mapped_column(
0648         UUID_TYPE,
0649         ForeignKey("documents.id", ondelete="CASCADE"),
0650         nullable=False,
```

```
0651 )
0652 collab_mode: Mapped[str] = mapped_column(
0653     String(16), nullable=False, default="open"
0654 )
0655 watermark_policy: Mapped[str] = mapped_column(
0656     String(16), nullable=False, default="optional"
0657 )
0658 export_policy: Mapped[str] = mapped_column(
0659     String(16), nullable=False, default="allow"
0660 )
0661 updated_at: Mapped[datetime] = mapped_column(
0662     TIMESTAMP(timezone=True),
0663     nullable=False,
0664     server_default=func.now(),
0665     onupdate=func.now(),
0666 )
0667
0668 def __repr__(self) -> str:
0669     return f"<PermissionConfig {self.doc_id} mode={self.collab_mode}>"
0670 # =====
0671 # FILE: backend/main.py
0672 # =====
0673 """
0674 FastAPI 应用入口
0675 =====
0676 - 配置 CORS 中间件
0677 - 挂载 REST API 路由
0678 - 挂载 Yjs WebSocket 协同端点 (/ws/{doc_id})
0679
0680 启动:
0681 uvicorn main:app --reload --host 0.0.0.0 --port 8000
0682 """
0683
0684 import uuid
0685 from contextlib import asynccontextmanager
0686
0687 from fastapi import FastAPI
0688 from fastapi.middleware.cors import CORSMiddleware
0689 from sqlalchemy import select
0690
0691 from api import agents as agents_api
0692 from api import documents as documents_api
0693 from api import literature as literature_api
0694 from api import watermark as watermark_api
0695 from config import settings
0696 from database import init_db
0697 from websocket.document_ws import collaborative_router
0698
0699
0700 @asynccontextmanager
```

```
0701 async def lifespan(app: FastAPI):
0702     """
0703     应用生命周期管理:
0704     - 启动: 初始化数据库表 (开发环境)
0705     - 关闭: 释放资源 (数据库连接池由 engine 自动管理)
0706     """
0707     # 启动时创建数据库表 (骨架阶段; 生产环境建议使用 Alembic)
0708     await init_db()
0709
0710     # 初始化演示数据 (用户 + 默认文档), 保证前端开箱即用
0711     await seed_demo_data()
0712     # 初始化文献种子语料 (调研 Agent 检索库)
0713     await seed_literature()
0714
0715     yield
0716
0717     # 关闭时清理 (预留)
0718     pass
0719
0720
0721     # -----
0722     # 演示数据初始化: 固定 UUID, 前端通过 /api/bootstrap 获取
0723     # -----
0724     DEMO_USER_ID = uuid.UUID("00000000-0000-4000-8000-000000000001")
0725     DEMO_DOC_ID = uuid.UUID("00000000-0000-4000-8000-0000000000a1")
0726
0727
0728     async def seed_demo_data() -> None:
0729         """若演示用户/文档不存在则创建 (幂等)"""
0730         from models import Document, User
0731         from database import async_session_factory
0732
0733         async with async_session_factory() as db:
0734             user = await db.get(User, DEMO_USER_ID)
0735             if user is None:
0736                 user = User(
0737                     id=DEMO_USER_ID,
0738                     username="demo-researcher",
0739                     email="demo@research.local",
0740                     display_name="演示研究员",
0741                     role="researcher",
0742                 )
0743                 db.add(user)
0744
0745             doc = await db.get(Document, DEMO_DOC_ID)
0746             if doc is None:
0747                 doc = Document(
0748                     id=DEMO_DOC_ID,
0749                     title="演示文档 · 多Agent协同科研编辑",
0750                     owner_id=DEMO_USER_ID,
```

```
0751 content="欢迎来到智溯协同编辑器。左侧可触发 Agent, 右侧可直接编辑。",
0752 )
0753 db.add(doc)
0754
0755 await db.commit()
0756
0757
0758 # -----
0759 # 文献种子语料: 覆盖水印算法 / CRDT 协同 / 多 Agent / 科研诚信等主题
0760 # -----
0761 _SEED_LITERATURE: list[dict] = [
0762 {
0763 "title": "A Watermark for Large Language Models",
0764 "authors": "Kirchenbauer J, Geiping J, Wen Y",
0765 "year": 2023,
0766 "source": "arXiv:2301.10226",
0767 "abstract": "提出基于绿名单的文本水印方案, 通过约束 LLM 解码过程在生成文本中嵌入可检测标记, 无需修改模型参数即可追溯 AI 生成内容来源。",
0768 "keywords": "watermark, large language model, AIGC, green list",
0769 "url": "https://arxiv.org/abs/2301.10226",
0770 },
0771 {
0772 "title": "Yjs: A Framework for Near Real-Time Peer-to-Peer Shared Editing on Arbitrary Data Types",
0773 "authors": "Jahns P",
0774 "year": 2021,
0775 "source": "arXiv:2102.12943",
0776 "abstract": "介绍 Yjs CRDT 框架: 基于冲突无关复制数据类型实现多端实时协同编辑, 无需中心服务器协调, 支持文本与富文本结构。",
0777 "keywords": "CRDT, Yjs, real-time collaboration, peer-to-peer",
0778 "url": "https://arxiv.org/abs/2102.12943",
0779 },
0780 {
0781 "title": "Multi-Agent Systems for Scientific Research: A Survey",
0782 "authors": "Wang L, Zhang X, Chen R",
0783 "year": 2024,
0784 "source": "arXiv:2403.07128",
0785 "abstract": "综述多智能体系统在科研场景的应用: 文献调研、实验设计、论文撰写与审稿等环节的自动化, 讨论协作框架与一致性保证。",
0786 "keywords": "multi-agent, LLM, scientific research, survey",
0787 "url": "https://arxiv.org/abs/2403.07128",
0788 },
0789 {
0790 "title": "Provenance for the Web: Trustworthy Computing Chains",
0791 "authors": "Moreau L, Groth P",
0792 "year": 2013,
0793 "source": "W3C PROV",
0794 "abstract": "提出基于哈希链与声明式规则的数据溯源模型, 为版权归属、内容演化与可信计算提供形式化基础。",
0795 "keywords": "provenance, hash chain, trust, authorship",
0796 "url": "https://www.w3.org/TR/prov-overview/",
0797 },
0798 {
0799 "title": "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks",
0800 "authors": "Lewis P, Perez E, Piktus A",
```



```
0801     "year": 2020,
0802     "source": "NeurIPS 2020",
0803     "abstract": "提出 RAG 框架: 结合检索器与生成器, 使模型从外部知识库检索证据支撑生成, 显著提升知识密集型任务的事实准确性。",
0804     "keywords": "RAG, retrieval, knowledge, generation",
0805     "url": "https://arxiv.org/abs/2005.11401",
0806 },
0807 {
0808     "title": "A Survey on Collaborative Editing Systems and CRDTs",
0809     "authors": "Nedelec B, Molli P, Mostefaoui A",
0810     "year": 2015,
0811     "source": "arXiv:1507.03967",
0812     "abstract": "系统梳理协同编辑技术演进: OT 与 CRDT 两大路线, 分析并发操作收敛性与离线编辑支持, 是分布式协同的经典综述。",
0813     "keywords": "collaborative editing, CRDT, OT, concurrency",
0814     "url": "https://arxiv.org/abs/1507.03967",
0815 },
0816 {
0817     "title": "On the Risks of LLM-Generated Content and Detection Methods",
0818     "authors": "Liu X, Wang Y, Li H",
0819     "year": 2024,
0820     "source": "arXiv:2402.04667",
0821     "abstract": "分析大模型生成内容在学术诚信场景中的风险 (造假、洗稿、代写), 综述文本水印、统计检测与元数据方案的能力边界。",
0822     "keywords": "AIGC, academic integrity, watermark, detection",
0823     "url": "https://arxiv.org/abs/2402.04667",
0824 },
0825 {
0826     "title": "Fast and Accurate Real-Time Collaboration with Yjs",
0827     "authors": "Jahns P",
0828     "year": 2022,
0829     "source": "Yjs Documentation",
0830     "abstract": "阐述 Yjs 的二进制同步协议 (y-websocket) 与状态快照机制, 面向工程实践的协同性能优化指南。",
0831     "keywords": "Yjs, y-websocket, sync protocol, binary",
0832     "url": "https://docs.yjs.dev/",
0833 },
0834 ]
0835
0836
0837 async def seed_literature() -> None:
0838     """若文献表为空则写入种子语料 (幂等)"""
0839     from models import Literature
0840     from database import async_session_factory
0841
0842     async with async_session_factory() as db:
0843         existing = (await db.execute(select(Literature).limit(1))).scalar_one_or_none()
0844         if existing is not None:
0845             return
0846         for item in _SEED_LITERATURE:
0847             db.add(Literature(**item))
0848         await db.commit()
0849
0850
```

```
0851 # -----
0852 # 创建应用
0853 # -----
0854 app = FastAPI(
0855     title=settings.APP_NAME,
0856     version=settings.APP_VERSION,
0857     description="面向科研诚信的多Agent实时协同与版权溯源系统",
0858     lifespan=lifespan,
0859 )
0860
0861 # -----
0862 # CORS 中间件: 允许前端跨域访问
0863 # -----
0864 app.add_middleware(
0865     CORSMiddleware,
0866     allow_origins=settings.CORS_ORIGINS,
0867     allow_credentials=True,
0868     allow_methods=["*"],
0869     allow_headers=["*"],
0870 )
0871
0872 # -----
0873 # 注册 REST API 路由
0874 # -----
0875 app.include_router(documents_api.router)
0876 app.include_router(agents_api.router)
0877 app.include_router(watermark_api.router)
0878 app.include_router(literature_api.router)
0879
0880 # -----
0881 # 注册 WebSocket 协同路由 (Yjs)
0882 # - WebSocket 端点: /ws/{doc_id}
0883 # - 同时也挂载 y-websocket 的 /yjs 心跳/就绪检查
0884 # -----
0885 app.include_router(collaborative_router)
0886
0887
0888 # -----
0889 # 健康检查
0890 # -----
0891 @app.get("/api/health")
0892 async def health_check() -> dict:
0893     """服务健康检查端点"""
0894     return {
0895         "status": "ok",
0896         "app": settings.APP_NAME,
0897         "version": settings.APP_VERSION,
0898     }
0899
0900
```

```
0901 @app.get("/api/bootstrap")
0902 async def bootstrap() -> dict:
0903     """前端初始化数据: 演示用户与文档 (前端启动时调用一次)"""
0904     from sqlalchemy import select
0905
0906     from database import async_session_factory
0907     from models import Document
0908
0909     async with async_session_factory() as db:
0910         docs = (
0911             await db.execute(select(Document).order_by(Document.updated_at.desc()))
0912             ).scalars().all()
0913         return {
0914             "demo_user_id": str(DEMO_USER_ID),
0915             "demo_doc_id": str(DEMO_DOC_ID),
0916             "documents": [
0917                 {
0918                     "id": str(d.id),
0919                     "title": d.title,
0920                     "owner_id": str(d.owner_id),
0921                     "content": d.content,
0922                     "watermark_status": d.watermark_status,
0923                     "created_at": d.created_at.isoformat(),
0924                     "updated_at": d.updated_at.isoformat(),
0925                 }
0926                 for d in docs
0927             ],
0928         }
0929
0930
0931 @app.get("/")
0932 async def root() -> dict:
0933     """根路径, 便于浏览器确认服务已启动"""
0934     return {
0935         "service": settings.APP_NAME,
0936         "docs": "/docs",
0937         "health": "/api/health",
0938         "websocket": "/ws/{doc_id}",
0939     }
0940 # =====
0941 # FILE: backend\api\__init__.py
0942 # =====
0943 """
0944 API 路由包
0945 =====
0946 包含:
0947 - documents.py : 文档 CRUD 与协同状态存取
0948 - agents.py : Agent 触发与状态查询
0949 - watermark.py : 水印检测与溯源
0950 """
```

```

0951 # =====
0952 # FILE: backend\api\agents.py
0953 # =====
0954 """
0955 Agent API
0956 =====
0957 提供 Agent 的触发、状态查询、会话消息接口。
0958 实际 LLM 调用与 LangGraph 编排位于 services/agent_orchestrator.py。
0959 此文件仅负责 HTTP 层对接。
0960 """
0961
0962 import uuid
0963 from typing import List, Optional
0964
0965 from fastapi import APIRouter, Depends, HTTPException
0966 from pydantic import BaseModel, Field
0967 from sqlalchemy.ext.asyncio import AsyncSession
0968
0969 from database import get_db
0970 from services.agent_orchestrator import (
0971     AgentType,
0972     OrchestratorService,
0973 )
0974
0975 router = APIRouter(prefix="/api/agents", tags=["agents"])
0976
0977 # 全局单例编排器 (跨请求保留会话, 供消息轮询接口查询)
0978 orchestrator = OrchestratorService()
0979
0980
0981 # -----
0982 # Pydantic Schemas
0983 # -----
0984 class AgentInvokeRequest(BaseModel):
0985     """触发 Agent 请求体"""
0986
0987     doc_id: uuid.UUID
0988     agent_type: AgentType
0989     instruction: str = Field(..., min_length=1, description="给 Agent 的具体指令")
0990
0991
0992 class AgentStateOut(BaseModel):
0993     """Agent 状态响应"""
0994
0995     type: AgentType
0996     status: str
0997     doc_id: Optional[uuid.UUID] = None
0998     current_step: Optional[str] = None
0999     started_at: Optional[str] = None
1000     finished_at: Optional[str] = None

```

```
1001
1002
1003 class AgentSessionOut(BaseModel):
1004     """Agent 会话摘要"""
1005
1006     session_id: uuid.UUID
1007     doc_id: uuid.UUID
1008     status: str
1009     created_at: str
1010
1011
1012 # -----
1013 # API 端点
1014 # -----
1015 @router.post("/invoke", status_code=202)
1016 async def invoke_agent(
1017     payload: AgentInvokeRequest,
1018 ) -> dict:
1019     """
1020     触发 ResearchAgent / WriterAgent / SupervisorAgent。
1021
1022     返回 202 Accepted:
1023     任务在后台异步运行 (FastAPI BackgroundTasks),
1024     结果通过 WebSocket `/ws/{doc_id}` 实时推送至前端。
1025     """
1026     # TODO: 接入 FastAPI BackgroundTasks 或 Celery 任务队列
1027     # 骨架实现: 直接调用编排服务 (同步演示, 模拟耗时 <1s)
1028     session_id: str = await orchestrator.start_session(
1029         doc_id=str(payload.doc_id),
1030         agent_type=payload.agent_type,
1031         instruction=payload.instruction,
1032     )
1033
1034     return {
1035         "accepted": True,
1036         "session_id": session_id,
1037         "message": f"{{payload.agent_type}} agent 已启动",
1038     }
1039
1040
1041 @router.get("/sessions/{session_id}/state", response_model=AgentStateOut)
1042 async def get_agent_state(
1043     session_id: uuid.UUID,
1044 ) -> dict:
1045     """查询 Agent 会话当前状态"""
1046     session = orchestrator.get_session(str(session_id))
1047     if session is None:
1048         return {
1049             "type": AgentType.SUPERVISOR,
1050             "status": "idle",
```

```

1051     "doc_id": None,
1052     "current_step": None,
1053     "started_at": None,
1054     "finished_at": None,
1055     }
1056     return {
1057         "type": session.get("agent_type", AgentType.SUPERVISOR),
1058         "status": session.get("status", "idle"),
1059         "doc_id": session.get("doc_id"),
1060         "current_step": None,
1061         "started_at": None,
1062         "finished_at": None,
1063     }
1064
1065
1066     @router.get("/sessions/{session_id}/messages")
1067     async def get_session_messages(
1068         session_id: uuid.UUID,
1069         limit: int = 50,
1070     ) -> dict:
1071         """
1072         获取 Agent 会话历史消息 (用于前端回放流式输出)。
1073         将编排器内部消息映射为前端 AgentMessage 结构 (camelCase)。
1074         """
1075         session = orchestrator.get_session(str(session_id))
1076         raw = (session or {}).get("state", {}).get("messages", [])
1077
1078         messages = [
1079             {
1080                 "id": f"{session_id}-{i}",
1081                 "sessionId": str(session_id),
1082                 "agentType": m.get("agent", "supervisor"),
1083                 "role": m.get("role", "agent"),
1084                 "content": m.get("content", ""),
1085                 "phase": "done",
1086                 "createdAt": None,
1087             }
1088             for i, m in enumerate(raw)
1089             ][:-limit]
1090
1091         return {
1092             "session_id": str(session_id),
1093             "messages": messages,
1094             "total": len(messages),
1095         }
1096     # =====
1097     # FILE: backend\api\documents.py
1098     # =====
1099     """
1100     文档 CRUD API

```

```
1101 =====
1102 提供文档的基本增删改查，以及 Yjs 协同状态快照的持久化接口。
1103 内容更新时自动写入操作日志 (哈希链)，供溯源链模块追溯。
1104 """
1105
1106 import base64
1107 import uuid
1108 from datetime import datetime
1109 from typing import List, Optional
1110
1111 from fastapi import APIRouter, Depends, HTTPException, Response, status
1112 from pydantic import BaseModel, Field
1113 from sqlalchemy import func as _sql_func
1114 from sqlalchemy import select, update
1115 from sqlalchemy.ext.asyncio import AsyncSession
1116
1117 from database import get_db
1118 from models import Document, OpLog
1119 from services.oplog_chain import OpLogHashChain, oplog_append_lock
1120
1121 router = APIRouter(prefix="/api/documents", tags=["documents"])
1122
1123 # -----
1124 # Pydantic Schemas
1125 # -----
1126
1127 class DocumentCreate(BaseModel):
1128     """创建文档请求体"""
1129
1130     title: str = Field(..., min_length=1, max_length=512)
1131     owner_id: uuid.UUID
1132     content: str = ""
1133
1134
1135 class DocumentUpdate(BaseModel):
1136     """更新文档请求体"""
1137
1138     title: Optional[str] = Field(None, min_length=1, max_length=512)
1139     content: Optional[str] = None
1140     yjs_state: Optional[bytes] = None # 以 base64 / 二进制传入的 Yjs 快照
1141     watermark_status: Optional[int] = Field(None, ge=0, le=2)
1142     operator_id: Optional[uuid.UUID] = None # 操作者 (写入溯源链日志用)
1143
1144
1145 class DocumentOut(BaseModel):
1146     """文档响应模型"""
1147
1148     id: uuid.UUID
1149     title: str
1150     owner_id: uuid.UUID
```

```
1151     content: str
1152     watermark_status: int
1153     agent_session_id: Optional[uuid.UUID] = None
1154     created_at: datetime
1155     updated_at: datetime
1156
1157     class Config:
1158         from_attributes = True
1159
1160
1161     # -----
1162     # API 端点
1163     # -----
1164     @router.get("", response_model=List[DocumentOut])
1165     async def list_documents(
1166         owner_id: Optional[uuid.UUID] = None,
1167         db: AsyncSession = Depends(get_db),
1168     ) -> List[Document]:
1169         """
1170         文档列表。可按 owner_id 过滤。
1171         """
1172         stmt = select(Document).order_by(Document.updated_at.desc())
1173         if owner_id:
1174             stmt = stmt.where(Document.owner_id == owner_id)
1175         result = await db.execute(stmt)
1176         return list(result.scalars().all())
1177
1178
1179     @router.post("", response_model=DocumentOut, status_code=status.HTTP_201_CREATED)
1180     async def create_document(
1181         payload: DocumentCreate,
1182         db: AsyncSession = Depends(get_db),
1183     ) -> Document:
1184         """创建新文档"""
1185         doc = Document(
1186             title=payload.title,
1187             owner_id=payload.owner_id,
1188             content=payload.content,
1189         )
1190         db.add(doc)
1191         await db.commit()
1192         await db.refresh(doc)
1193         return doc
1194
1195
1196     @router.get("/{doc_id}", response_model=DocumentOut)
1197     async def get_document(
1198         doc_id: uuid.UUID,
1199         db: AsyncSession = Depends(get_db),
1200     ) -> Document:
```



```
1201 """获取单个文档"""
1202 doc = await db.get(Document, doc_id)
1203 if doc is None:
1204     raise HTTPException(status_code=404, detail="Document not found")
1205     return doc
1206
1207
1208 @router.patch("/{doc_id}", response_model=DocumentOut)
1209 async def update_document(
1210     doc_id: uuid.UUID,
1211     payload: DocumentUpdate,
1212     db: AsyncSession = Depends(get_db),
1213 ) -> Document:
1214     """更新文档 (标题 / 内容 / 协同状态快照 / 水印状态)"""
1215     # 串行化本文档的溯源链追加: 并发 PATCH 读到相同链尾会算出相同
1216     # current_hash, 违反 op_logs.current_hash UNIQUE 约束 -> 500 + 断链
1217     # (缺陷 D1, 见 oplog_chain.oplog_append_lock 说明)。锁覆盖
1218     # 「读文档 -> 写 OpLog -> commit」 全过程, 提交后释放。
1219     async with oplog_append_lock(doc_id):
1220         doc = await db.get(Document, doc_id)
1221         if doc is None:
1222             raise HTTPException(status_code=404, detail="Document not found")
1223
1224         data = payload.model_dump(exclude_unset=True)
1225         # operator_id 是溯源链元数据, 不属于 Document 字段
1226         operator_id = data.pop("operator_id", None) or doc.owner_id
1227
1228         content_changed = "content" in data and data["content"] != doc.content
1229         old_content = doc.content
1230
1231         for field, value in data.items():
1232             setattr(doc, field, value)
1233
1234         # 内容变更 -> 写入溯源链操作日志 (哈希链)
1235         if content_changed:
1236             await _append_content_log(db, doc, operator_id, old_content, doc.content)
1237
1238         await db.commit()
1239         await db.refresh(doc)
1240         return doc
1241
1242
1243 async def _append_content_log(
1244     db: AsyncSession,
1245     doc: Document,
1246     operator_id: uuid.UUID,
1247     old_content: str,
1248     new_content: str,
1249 ) -> None:
1250     """
```

```
1251 内容更新时追加一条溯源链日志:
1252 - 取该文档链尾 current_hash 作为 prev_hash;
1253 - current_hash 由 flush 后的数据库时间戳计算, 保证与
1254 verify_provenance 的校验规则一致。
1255 """
1256 from models import OpLog as OpLogModel # noqa: F401 (类型引用)
1257
1258 last_stmt = (
1259     select(OpLog)
1260     .where(OpLog.doc_id == doc.id)
1261     .order_by(OpLog.created_at.desc())
1262     .limit(1)
1263 )
1264 last = (await db.execute(last_stmt)).scalar_one_or_none()
1265 prev_hash = last.current_hash if last else ""
1266
1267 # 首次内容写入视为 insert, 后续修改视为 replace
1268 op_type = "insert" if not old_content.strip() else "replace"
1269 operation = {
1270     "content_len": len(new_content),
1271     "content_head": new_content[:200],
1272     "old_len": len(old_content),
1273     "delta": len(new_content) - len(old_content),
1274 }
1275
1276 log = OpLog(
1277     doc_id=doc.id,
1278     user_id=operator_id,
1279     op_type=op_type,
1280     operation=operation,
1281     prev_hash=prev_hash,
1282     current_hash="", # flush 后按数据库时间戳补算
1283 )
1284 db.add(log)
1285 await db.flush()
1286 log.current_hash = OpLogHashChain.compute_hash(
1287     prev_hash=prev_hash,
1288     operation=operation,
1289     timestamp=log.created_at,
1290 )
1291
1292
1293 @router.delete("/{doc_id}", status_code=status.HTTP_204_NO_CONTENT)
1294 async def delete_document(
1295     doc_id: uuid.UUID,
1296     db: AsyncSession = Depends(get_db),
1297 ) -> None:
1298     """删除文档"""
1299     doc = await db.get(Document, doc_id)
1300     if doc is None:
```

```
1301     raise HTTPException(status_code=404, detail="Document not found")
1302     await db.delete(doc)
1303     await db.commit()
1304
1305
1306     # -----
1307     # 文档导出 (Markdown + 溯源元数据)
1308     # -----
1309     @router.get("/{doc_id}/export")
1310     async def export_document(
1311         doc_id: uuid.UUID,
1312         db: AsyncSession = Depends(get_db),
1313     ) -> Response:
1314         """
1315         导出文档为 Markdown 文本, 并附带完整的溯源元数据:
1316         - 文档标题 / 更新时间 / 字数统计
1317         - 正文内容 (段落保留)
1318         - 溯源哈希链校验状态 (valid / 日志条数)
1319         - 水印检测记录
1320         - 可引用的参考文献列表
1321
1322         前端可直接下载为 .md 文件, 或用于软著演示的"版权可信导出"。
1323         """
1324         from models import Literature, WatermarkRecord
1325
1326         doc = await db.get(Document, doc_id)
1327         if doc is None:
1328             raise HTTPException(status_code=404, detail="Document not found")
1329
1330         # 溯源链统计与校验
1331         oplog_stmt = (
1332             select(OpLog).where(OpLog.doc_id == doc_id).order_by(OpLog.created_at.asc())
1333         )
1334         logs = list((await db.execute(oplog_stmt)).scalars().all())
1335         chain_ok, checked = True, 0
1336         prev_hash = ""
1337         for log in logs:
1338             if log.prev_hash != prev_hash:
1339                 chain_ok = False
1340                 break
1341             expected = OpLogHashChain.compute_hash(
1342                 prev_hash=log.prev_hash,
1343                 operation=log.operation,
1344                 timestamp=log.created_at,
1345             )
1346             if log.current_hash != expected:
1347                 chain_ok = False
1348                 break
1349             prev_hash = log.current_hash
1350             checked += 1
```

```
1351
1352 # 水印记录
1353 wm_stmt = select(WatermarkRecord).where(WatermarkRecord.doc_id == doc_id)
1354 wm_records = list((await db.execute(wm_stmt)).scalars().all())
1355
1356 # 参考文献 (全库, 供引用参考)
1357 lit_stmt = select(Literature).order_by(Literature.year.desc()).limit(10)
1358 lits = list((await db.execute(lit_stmt)).scalars().all())
1359
1360 lines: List[str] = []
1361 lines.append(f"# {doc.title}")
1362 lines.append("")
1363 lines.append(f"> 导出时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
1364 lines.append(f"> 文档字数: {len(doc.content)} | 操作日志: {len(logs)} 条")
1365 lines.append(f"> 溯源哈希链校验: {' 完整可信' if chain_ok else ' 校验失败'} (检查 {checked} 条)")
1366 lines.append("")
1367 lines.append("---")
1368 lines.append("")
1369
1370 # 正文 (剥离编辑器 HTML 标签, 保留段落)
1371 import re as _re
1372
1373 content = doc.content or "(空文档)"
1374 content = _re.sub(r"<[^>]+>", "", content)
1375 if content.strip():
1376     for para in content.split("\n"):
1377         para = para.strip()
1378         if not para:
1379             continue
1380         lines.append(para)
1381         lines.append("")
1382     else:
1383         lines.append("(空文档)")
1384         lines.append("")
1385
1386 # 水印记录
1387 if wm_records:
1388     lines.append("---")
1389     lines.append("")
1390     lines.append("##      水印检测记录")
1391     lines.append("")
1392     for i, r in enumerate(wm_records, 1):
1393         lines.append(
1394             f"{i}. {r.created_at.strftime('%Y-%m-%d %H:%M')} · "
1395             f"模型 {r.model_name} · gamma={r.gamma} · delta={r.delta}"
1396         )
1397     lines.append("")
1398
1399 # 参考文献
1400 if lits:
```

```

1401     lines.append("---")
1402     lines.append("")
1403     lines.append("##      参考文献 (GB/T 7714)")
1404     lines.append("")
1405     for i, l in enumerate(lits, 1):
1406         cite = f"{l.authors}. {l.title} [{l.source}]. {l.year}."
1407         lines.append(f"[{i}] {cite}")
1408     lines.append("")
1409
1410     body = "\n".join(lines)
1411     filename = f'{doc.title or "document"}.md'
1412     return Response(
1413         content=body,
1414         media_type="text/markdown; charset=utf-8",
1415         headers={
1416             "Content-Disposition": f'attachment; filename*=UTF-8\'\'{_urlquote(filename)}'
1417         },
1418     )
1419
1420
1421     def _urlquote(name: str) -> str:
1422         """URL 编码文件名 (RFC 5987, 支持中文)"""
1423         from urllib.parse import quote
1424
1425         return quote(name)
1426
1427
1428     # -----
1429     # Yjs 协同状态持久化
1430     # -----
1431     @router.get("/{doc_id}/yjs-state")
1432     async def get_yjs_state(
1433         doc_id: uuid.UUID,
1434         db: AsyncSession = Depends(get_db),
1435     ) -> dict:
1436         """
1437         获取文档 Yjs CRDT 状态快照 (用于新加入的客户端离线初始化)。
1438         返回 base64 编码字符串。
1439         """
1440         doc = await db.get(Document, doc_id)
1441         if doc is None:
1442             raise HTTPException(status_code=404, detail="Document not found")
1443
1444         state_b64 = base64.b64encode(doc.yjs_state or b'').decode("ascii")
1445         return {"doc_id": str(doc_id), "yjs_state_b64": state_b64}
1446
1447
1448     @router.put("/{doc_id}/yjs-state")
1449     async def save_yjs_state(
1450         doc_id: uuid.UUID,

```

```
1451     payload: dict,
1452     db: AsyncSession = Depends(get_db),
1453 ) -> dict:
1454     """
1455     保存文档 Yjs 状态快照。
1456     请求体: {"yjs_state_b64": "..."} (编码后的 Yjs Update Message)
1457     """
1458     state_b64: str = payload.get("yjs_state_b64", "")
1459     try:
1460         state_bytes = base64.b64decode(state_b64)
1461     except Exception:
1462         raise HTTPException(status_code=400, detail="Invalid base64")
1463
1464     stmt = (
1465         update(Document)
1466         .where(Document.id == doc_id)
1467         .values(yjs_state=state_bytes, updated_at=func_now_sql())
1468     )
1469     result = await db.execute(stmt)
1470     if result.rowcount == 0:
1471         raise HTTPException(status_code=404, detail="Document not found")
1472     await db.commit()
1473     return {"ok": True}
1474
1475
1476     # 辅助: 直接使用 SQL.now()
1477     def func_now_sql():
1478         return _sql_func.now()
1479
1480     # =====
1481     # FILE: backend\api\literature.py
1482     # =====
1483     """
1484     文献检索 API
1485     =====
1486     面向调研 (Research) Agent 的文献资源检索端点。
1487
1488     - 数据源: literature 表 (种子语料 + 人工导入)
1489     - 检索策略: 标题/摘要/关键词的简单包含匹配 (骨架阶段;
1490       后续可接入 ArXiv / 知网 / 语义 Scholar 等外部检索源)
1491     - 能力: 关键词检索 + 文献详情 + 插入引文格式文本
1492     """
1493
1494     import uuid
1495     from typing import List, Optional
1496
1497     from fastapi import APIRouter, Depends, Query
1498     from pydantic import BaseModel, Field
1499     from sqlalchemy import or_, select
1500     from sqlalchemy.ext.asyncio import AsyncSession
```

```
1501 const session = useMemo(() => getCollabSession(docId), [docId]);
1502 const provider: WebsocketProvider = session.provider;
1503
1504 // ---- 监听连接状态 / 同步 / 在线人数 (不销毁 provider) ----
1505 useEffect(() => {
1506   const statusListener = (event: { status: string }) => {
1507     setIsConnected(event.status === 'connected');
1508   };
1509   const syncListener = (isSynced: boolean) => {
1510     if (isSynced) {
1511       // 同步完成, 触发一次作者统计
1512       const a = collectAuthors(ydoc);
1513       authorsRef.current = a;
1514       setAuthors(new Set(a));
1515     }
1516   };
1517   const awarenessListener = () => {
1518     setOnlineCount(provider.awareness.getStates().size);
1519   };
1520
1521   provider.on('status', statusListener);
1522   provider.on('sync', syncListener);
1523   provider.awareness.on('change', awarenessListener);
1524
1525   return () => {
1526     provider.off('status', statusListener);
1527     provider.off('sync', syncListener);
1528     provider.awareness.off('change', awarenessListener);
1529   };
1530   // eslint-disable-next-line react-hooks/exhaustive-deps
1531 }, [provider]);
1532
1533 // ---- Tiptap 编辑器 ----
1534 const editor = useEditor({
1535   extensions: [
1536     StarterKit.configure({
1537       history: false, // 协同场景由 Y.UndoManager 管理撤销
1538     }),
1539     AuthorMark,
1540     Collaboration.configure({
1541       fragment: ydoc.getXmlFragment('default'),
1542     }),
1543     CollaborationCursor.configure({
1544       provider,
1545       user: {
1546         name: username,
1547         color: '#4f46e5',
1548       },
1549     }),
1550   ],
1551   // 自定义扩展: 需要读取 ydoc 实例 (通过 args 传入)
```

```
1551   AuthorHighlight.configure({ ydoc }),
1552   ],
1553   content: "",
1554   editorProps: {
1555     attributes: {
1556       class: 'tiptap',
1557     },
1558   },
1559   });
1560
1561   // ---- 将 editor 注册到协同会话 (供 Agent 面板以本地事务方式插入内容) ----
1562   // StrictMode double-mount 会创建/销毁两个 editor, 注册时用最新实例;
1563   // 清理时仅当前注册的仍是本实例才置空, 避免误清后续注册的编辑器。
1564   useEffect(() => {
1565     if (!editor) return;
1566     session.editor = editor;
1567     return () => {
1568       if (session.editor === editor) {
1569         session.editor = null;
1570       }
1571     };
1572   }, [editor, session]);
1573
1574   // ---- 初始内容加载: 将后端已保存内容写入 Yjs (打开文档即显示) ----
1575   // 条件:
1576   // - editor 就绪且后端有内容;
1577   // - fragment 仍为空 (Tiptap 初始化的空段落不算) 或未加载过,
1578   // 避免覆盖远端协同内容与 StrictMode 重复加载。
1579   useEffect(() => {
1580     if (!editor || !initialContent || session.loaded) return;
1581     const fragment = ydoc.getXmlFragment('default');
1582     const isEmpty =
1583       fragment.length === 0 ||
1584       (fragment.length === 1 &&
1585         fragment.get(0) instanceof Y.XmlElement &&
1586         (fragment.get(0) as Y.XmlElement).length === 0);
1587     if (!isEmpty) return;
1588
1589     session.loaded = true;
1590     // 经 PM 通道写入, 与 AI 插入同一路径 (ySyncPlugin 兼容)
1591     editor.commands.setContent(initialContent);
1592   }, [editor, initialContent, ydoc, session]);
1593
1594   // ---- 编辑持久化: 内容变化防抖 1.5s 后保存到后端 (并写入溯源链日志) ----
1595   useEffect(() => {
1596     if (!editor || !operatorId) return;
1597     let timer: number | undefined;
1598     const persist = () => {
1599       window.clearTimeout(timer);
1600       timer = window.setTimeout(() => {
```



```
1601 const text = getPlainText(ydoc);
1602 if (!text.trim()) return; // 空文档不落库
1603 updateDocument(docId, { content: text, operatorId }).catch((e) =>
1604 console.warn('[collab] 文档保存失败:', e)
1605 );
1606 }, 1500);
1607 };
1608 editor.on('update', persist);
1609 return () => {
1610 window.clearTimeout(timer);
1611 editor.off('update', persist);
1612 };
1613 }, [editor, docId, operatorId, ydoc]);
1614
1615 // ---- 监听远程更新 -> 刷新作者着色与统计 ----
1616 useEffect(() => {
1617 if (!editor) return;
1618
1619 // 本地输入: 标记为人类作者 (默认)
1620 const handleDocUpdate = () => {
1621 const fragment = ydoc.getXmlFragment('default');
1622 // 将用户本地插入的文本标记为 human
1623 // NOTE: 真实场景在 beforeinput/transaction 时给文本加 author=human 属性
1624 // 此处用 Y.XmlText.format 模拟: 对最新段落文本打标
1625 const texts: Y.XmlText[] = [];
1626 const walk = (node: Y.XmlElement | Y.XmlText | Y.XmlFragment): void => {
1627 if (node instanceof Y.XmlText) texts.push(node);
1628 else node.forEach(walk);
1629 };
1630 walk(fragment);
1631 for (const t of texts) {
1632 if (t.length > 0 && !t.getAttribute('author')) {
1633 // 默认全部标记为人类 (AI 内容由 appendAiText 携带 author mark 写入)
1634 // 属性值用 mark attrs 对象格式, 与 y-prosemirror 的 mark 映射一致
1635 t.setAttribute('author', { author: defaultAuthor });
1636 }
1637 }
1638 // 刷新统计
1639 const a = collectAuthors(ydoc);
1640 authorsRef.current = a;
1641 setAuthors(new Set(a));
1642 };
1643
1644 // Tiptap transaction 触发 (含本地输入与远程 CRDT 同步)
1645 editor.on('transaction', handleDocUpdate);
1646
1647 return () => {
1648 editor.off('transaction', handleDocUpdate);
1649 };
1650 // eslint-disable-next-line react-hooks/exhaustive-deps
```

```

1651   }, [editor, ydoc, defaultAuthor]);
1652
1653   return (
1654     <div
1655       className={flex flex-col bg-white rounded-lg shadow-sm overflow-hidden ${className}}
1656     >
1657       {/* 工具栏 */}
1658       <div className="flex items-center gap-2 px-4 py-2 border-b border-gray-200 bg-gray-50 text-xs">
1659         <span
1660           className={inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full ${
1661             isConnected
1662               ? 'bg-green-100 text-green-700'
1663               : 'bg-amber-100 text-amber-700'
1664             }}
1665         >
1666           <span
1667             className={w-1.5 h-1.5 rounded-full ${
1668               isConnected ? 'bg-green-500' : 'bg-amber-500 animate-pulse'
1669             }}
1670           />
1671           {isConnected ? '已连接' : '连接中...'}
1672         </span>
1673
1674         <span className="text-gray-400"> · </span>
1675         <span className="text-gray-500">{onlineCount} 人在线</span>
1676
1677         <span className="ml-auto flex flex-wrap items-center gap-2">
1678           {/* AI / 人类内容标识 */}
1679           <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded bg-blue-50 text-blue-600">
1680             <span className="w-2 h-2 rounded-sm bg-blue-400 inline-block" />
1681             AI 生成 {authors.has('ai') ? '√' : ''}
1682           </span>
1683           <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded bg-gray-100 text-gray-600">
1684             <span className="w-2 h-2 rounded-sm bg-white border border-gray-300 inline-block" />
1685             人类输入 {authors.has('human') ? '√' : ''}
1686           </span>
1687         </span>
1688       </div>
1689
1690       {/* 编辑器主体 */}
1691       <div className="flex-1 overflow-y-auto">
1692         <EditorContent editor={editor} />
1693       </div>
1694     </div>
1695   );
1696 }
1697 # =====
1698 # FILE: frontend\src\components\Literature\LiteraturePanel.tsx
1699 # =====
1700 import { useEffect, useState } from 'react';

```

```
1701 import type * as Y from 'yjs';
1702 import { searchLiterature, getCitation, type LiteratureItem } from '../lib/api';
1703 import { appendText } from '../lib/yjs';
1704 import { getCollabSession } from '../lib/collab';
1705
1706 /**
1707  * 文献检索面板
1708  * -----
1709  * 功能:
1710  * 1. 关键词检索文献语料库 (后端 literature 表)
1711  * 2. 展示标题 / 作者 / 年份 / 来源 / 摘要
1712  * 3. 一键将 GB/T 7714 引文插入协作文档末尾 (标记为人类输入)
1713  */
1714 interface LiteraturePanelProps {
1715   docId: string;
1716   ydoc: Y.Doc;
1717 }
1718
1719 export function LiteraturePanel({ docId, ydoc }: LiteraturePanelProps) {
1720   const [query, setQuery] = useState('');
1721   const [items, setItems] = useState<LiteratureItem[]>([]);
1722   const [loading, setLoading] = useState(false);
1723   const [error, setError] = useState<string | null>(null);
1724   const [inserting, setInserting] = useState<string | null>(null);
1725
1726   // 初始加载最近文献
1727   useEffect(() => {
1728     searchLiterature('')
1729     .then(setItems)
1730     .catch(() => setError('文献库加载失败'));
1731   }, []);
1732
1733   const handleSearch = async () => {
1734     setLoading(true);
1735     setError(null);
1736     try {
1737       const results = await searchLiterature(query);
1738       setItems(results);
1739     } catch (e) {
1740       setError(e instanceof Error ? e.message : '检索失败');
1741     } finally {
1742       setLoading(false);
1743     }
1744   };
1745
1746   const handleInsert = async (item: LiteratureItem) => {
1747     setInserting(item.id);
1748     try {
1749       const { citation } = await getCitation(item.id);
1750       const ok = appendText(
```

```

1751 getCollabSession(docId).editor,
1752 ydoc,
1753 `【引用】 ${citation}`,
1754 'human'
1755 );
1756 if (!ok) {
1757   setError('编辑器未就绪, 插入失败');
1758 }
1759 } catch (e) {
1760   setError(e instanceof Error ? e.message : '插入失败');
1761 } finally {
1762   setInserting(null);
1763 }
1764 };
1765
1766 return (
1767   <div className="space-y-4">
1768     <div>
1769       <h3 className="text-sm font-semibold text-gray-800"> 文献检索</h3>
1770       <p className="text-[11px] text-gray-400 mt-0.5">
1771         调研语料库 · 支持一键插入引文到文档
1772       </p>
1773     </div>
1774
1775     {/* 检索输入 */}
1776     <div className="flex gap-2">
1777       <input
1778         value={query}
1779         onChange={(e) => setQuery(e.target.value)}
1780         onKeyDown={(e) => e.key === 'Enter' && handleSearch()}
1781         placeholder="关键词, 如水印 / CRDT / 多Agent..."
1782         className="flex-1 min-w-0 text-xs px-2.5 py-1.5 rounded-md border border-gray-300 focus:outline-none focus:ring-2 focus:ring-blue-200"
1783       />
1784       <button
1785         onClick={handleSearch}
1786         disabled={loading}
1787         className="text-xs px-3 py-1.5 rounded-md bg-indigo-600 text-white hover:bg-indigo-500 disabled:opacity-40 transition-colors whitespace-nowrap"
1788       >
1789         {loading ? '检索中...' : '检索'}
1790       </button>
1791     </div>
1792
1793     {error && (
1794       <div className="text-[11px] text-red-500 bg-red-50 rounded px-2 py-1.5">
1795         ⚠ {error}
1796       </div>
1797     )}
1798
1799     {/* 检索结果 */}
1800     <div className="space-y-2.5">

```

```
1801 {items.length === 0 && !loading && (
1802 <p className="text-[11px] text-gray-400 text-center py-4">
1803 暂无文献, 输入关键词检索
1804 </p>
1805 )}
1806 {items.map((item) => (
1807 <div
1808   key={item.id}
1809   className="rounded-lg border border-gray-200 bg-gray-50/60 p-3 space-y-1.5"
1810 >
1811   <div className="text-xs font-medium text-gray-800 leading-snug">
1812     {item.title}
1813   </div>
1814   <div className="text-[10px] text-gray-400">
1815     {item.authors} · {item.year || '未知年份'} · {item.source}
1816   </div>
1817   <p className="text-[11px] text-gray-500 leading-relaxed line-clamp-3">
1818     {item.abstract}
1819   </p>
1820   <div className="flex items-center justify-between">
1821     <span className="text-[10px] text-gray-300 truncate max-w-[55%]">
1822       {item.keywords}
1823     </span>
1824     <button
1825       onClick={() => handleInsert(item)}
1826       disabled={inserting === item.id}
1827       className="text-[11px] px-2 py-1 rounded bg-white border border-indigo-200 text-indigo-600 hover:bg-indigo-50 disabled:opacity-40 transition-colors
whitespace-nowrap"
1828     >
1829       {inserting === item.id ? '插入中...' : '插入引用'}
1830     </button>
1831   </div>
1832 </div>
1833 )})}
1834 </div>
1835 </div>
1836 );
1837 }
1838
1839 # =====
1840 # FILE: frontend\src\components\Provenance\ProvenancePanel.tsx
1841 # =====
1842 import { useCallback, useEffect, useState } from 'react';
1843 import { getProvenance, verifyProvenance, type ProvenanceEntry } from '../lib/api';
1844
1845 /**
1846  * 版权溯源面板
1847  * -----
1848  * 展示文档操作日志哈希链 (OpLog Chain):
1849  * 1. 按时间正序列出全部操作 (类型 / 摘要 / 哈希缩略)
1850  * 2. 一键校验链条完整性 (防篡改验证)
```

```
1851  */
1852
1853  const OP_TYPE_META: Record<string, { label: string; cls: string }> = {
1854    insert: { label: '插入', cls: 'bg-gray-100 text-gray-600' },
1855    delete: { label: '删除', cls: 'bg-amber-50 text-amber-600' },
1856    replace: { label: '替换', cls: 'bg-purple-50 text-purple-600' },
1857    ai_generate: { label: 'AI 生成', cls: 'bg-blue-50 text-blue-600' },
1858    watermark_checked: { label: '水印检测', cls: 'bg-cyan-50 text-cyan-600' },
1859  };
1860
1861  interface ProvenancePanelProps {
1862    docId: string;
1863  }
1864
1865  export function ProvenancePanel({ docId }: ProvenancePanelProps) {
1866    const [entries, setEntries] = useState<ProvenanceEntry[]>([]);
1867    const [verify, setVerify] = useState<{ valid: boolean; checked: number } | null>(null);
1868    const [loading, setLoading] = useState(false);
1869    const [error, setError] = useState<string | null>(null);
1870
1871    const refresh = useCallback(async () => {
1872      setLoading(true);
1873      setError(null);
1874      try {
1875        const [list, v] = await Promise.all([
1876          getProvenance(docId),
1877          verifyProvenance(docId),
1878        ]);
1879        setEntries(list);
1880        setVerify(v);
1881      } catch (e) {
1882        setError(e instanceof Error ? e.message : '加载溯源链失败');
1883      } finally {
1884        setLoading(false);
1885      }
1886    }, [docId]);
1887
1888    useEffect(() => {
1889      refresh();
1890    }, [refresh]);
1891
1892    return (
1893      <div className="space-y-4">
1894        <div className="flex items-start justify-between">
1895          <div>
1896            <h3 className="text-sm font-semibold text-gray-800">      版权溯源链</h3>
1897            <p className="text-[11px] text-gray-400 mt-0.5">
1898              操作日志哈希链 · 防篡改审计
1899            </p>
1900          </div>
```

```
1901 <button
1902 onClick={refresh}
1903 disabled={!loading}
1904 className="text-[11px] px-2 py-1 rounded border border-gray-300 text-gray-500 hover:bg-gray-50 disabled:opacity-40 transition-colors"
1905 >
1906 {loading ? '...' : ' 刷新'}
1907 </button>
1908 </div>
1909
1910 {error && (
1911 <div className="text-[11px] text-red-500 bg-red-50 rounded px-2 py-1.5">
1912 ⚠ {error}
1913 </div>
1914 )}
1915
1916 {/* 完整性校验徽标 */}
1917 {verify && (
1918 <div
1919 className={flex items-center justify-between rounded-lg px-3 py-2 text-xs ${
1920 verify.valid
1921 ? 'bg-green-50 text-green-700 border border-green-200'
1922 : 'bg-red-50 text-red-700 border border-red-200'}
1923 )}
1924 >
1925 <span className="font-medium">
1926 {verify.valid ? '✓ 链条完整 · 未被篡改' : ' 检测到篡改 ! '}
1927 </span>
1928 <span className="text-[11px] opacity-70">{verify.checked} 条记录</span>
1929 </div>
1930 )}
1931
1932 {/* 链条目列表 */}
1933 {entries.length === 0 && !loading && !error && (
1934 <div className="text-[11px] text-gray-400 text-center py-6 border border-dashed border-gray-200 rounded-lg">
1935 暂无操作记录
1936 <br />
1937 <span className="text-gray-300">
1938 编辑文档或触发 Agent 后产生审计日志 (骨架阶段由后端写入)
1939 </span>
1940 </div>
1941 )}
1942
1943 <div className="space-y-2">
1944 {entries.map((e, i) => {
1945 const meta = OP_TYPE_META[e.op_type] ?? {
1946 label: e.op_type,
1947 cls: 'bg-gray-100 text-gray-600',
1948 };
1949 return (
1950 <div
```

```
1951     key={e.id}
1952     className="relative bg-white border border-gray-100 rounded-lg p-2.5 hover:border-gray-200 transition-colors"
1953     >
1954     { /* 链节点连线 */ }
1955     {i < entries.length - 1 && (
1956     <span className="absolute left-6 -bottom-2 w-px h-2 bg-gray-200" />
1957     )}
1958     <div className="flex items-center gap-2">
1959     <span className="w-1.5 h-1.5 rounded-full bg-indigo-400 shrink-0" />
1960     <span className={`text-[10px] font-medium px-1.5 py-0.5 rounded ${meta.cls}`}>
1961     {meta.label}
1962     </span>
1963     <span className="ml-auto text-[10px] text-gray-300">
1964     {new Date(e.created_at).toLocaleString('zh-CN', {
1965     month: '2-digit',
1966     day: '2-digit',
1967     hour: '2-digit',
1968     minute: '2-digit',
1969     })}
1970     </span>
1971     </div>
1972     <div className="mt-1.5 font-mono text-[10px] text-gray-400 break-all">
1973     hash: {e.current_hash.slice(0, 20)}...
1974     </div>
1975     </div>
1976     );
1977     }}
1978     </div>
1979     </div>
1980     );
1981     }
1982
1983     # =====
1984     # FILE: frontend\src\components\Watermark\WatermarkPanel.tsx
1985     # =====
1986     import { useCallback, useEffect, useState } from 'react';
1987     import type { WatermarkDetectionResult } from '@shared/types';
1988     import {
1989     detectDocumentWatermark,
1990     detectWatermark,
1991     getWatermarkRecords,
1992     type WatermarkRecordItem,
1993     } from '../lib/api';
1994
1995     /**
1996     * 水印检测面板
1997     * -----
1998     * 功能:
1999     * 1. 检测整个协作文档的文本是否包含 Kirchenbauer AI 水印,
2000     * 检测动作会写入 WatermarkRecord 与溯源链日志 (可追溯)
```



```
2001 * 2. 支持粘贴任意文本进行检测
2002 * 3. 可视化置信度 (进度条 + 判定徽标)
2003 * 4. 展示该文档的水印检测历史记录
2004 */
2005 interface WatermarkPanelProps {
2006   docId: string;
2007   getDocText: () => string;
2008 }
2009
2010 export function WatermarkPanel({ docId, getDocText }: WatermarkPanelProps) {
2011   const [customText, setCustomText] = useState("");
2012   const [result, setResult] = useState<WatermarkDetectionResult | null>(null);
2013   const [records, setRecords] = useState<WatermarkRecordItem[]>([]);
2014   const [loading, setLoading] = useState(false);
2015   const [error, setError] = useState<string | null>(null);
2016
2017   const refreshRecords = useCallback(async () => {
2018     try {
2019       const data = await getWatermarkRecords(docId);
2020       setRecords(data.records);
2021     } catch {
2022       // 历史记录加载失败不阻断主流程
2023     }
2024   }, [docId]);
2025
2026   useEffect(() => {
2027     refreshRecords();
2028   }, [refreshRecords]);
2029
2030   const runDetect = async (text: string, persist: boolean) => {
2031     if (!text.trim()) {
2032       setError('没有可检测的文本');
2033       return;
2034     }
2035     setError(null);
2036     setLoading(true);
2037     try {
2038       const started = performance.now();
2039       const r = persist
2040         ? await detectDocumentWatermark(docId)
2041         : await detectWatermark(text);
2042       setResult({ ...r, latencyMs: Math.round(performance.now() - started) });
2043       if (persist) {
2044         await refreshRecords();
2045       }
2046     } catch (e) {
2047       setError(e instanceof Error ? e.message : '检测失败');
2048     } finally {
2049       setLoading(false);
2050     }
  }
```

```

2051   };
2052
2053   const confidencePct = result ? Math.round(result.confidence * 100) : 0;
2054
2055   return (
2056     <div className="space-y-4">
2057       <div>
2058         <h3 className="text-sm font-semibold text-gray-800">          AIGC 水印检测</h3>
2059         <p className="text-[11px] text-gray-400 mt-0.5">
2060           Kirchenbauer 绿名单算法 · 判定文本是否为 AI 生成
2061         </p>
2062       </div>
2063
2064       {/* 操作按钮 */}
2065       <div className="grid grid-cols-1 gap-2">
2066         <button
2067           onClick={() => runDetect(getDocText(), true)}
2068           disabled={loading}
2069           className="w-full text-xs font-medium py-2 rounded-md bg-blue-600 text-white hover:bg-blue-500 disabled:opacity-40 disabled:cursor-not-allowed
2070           transition-colors"
2071           >
2072           {loading ? '检测中...' : '检测当前文档全文并留痕'}
2073         </button>
2074       </div>
2075
2076       {/* 自定义文本检测 */}
2077       <div>
2078         <textarea
2079           value={customText}
2080           onChange={(e) => setCustomText(e.target.value)}
2081           placeholder="或粘贴待检测文本..."
2082           rows={3}
2083           className="w-full text-xs p-2.5 rounded-md border border-gray-300 focus:outline-none focus:ring-2 focus:ring-blue-200 resize-none"
2084         />
2085         <button
2086           onClick={() => runDetect(customText, false)}
2087           disabled={loading || !customText.trim()}
2088           className="mt-1.5 w-full text-xs py-1.5 rounded-md border border-gray-300 text-gray-600 hover:bg-gray-50 disabled:opacity-40
2089           disabled:cursor-not-allowed transition-colors"
2090         >
2091           检测粘贴的文本
2092         </button>
2093       </div>
2094
2095       {error && (
2096         <div className="text-[11px] text-red-500 bg-red-50 rounded px-2 py-1.5">
2097           ⚠ {error}
2098         </div>
2099       )}
2100
2101       {/* 检测结果 */}
2102       {result && (

```

```
2101 <div className="rounded-lg border border-gray-200 bg-gray-50/60 p-3 space-y-3">
2102 <div className="flex items-center justify-between">
2103 <span className="text-xs text-gray-500">判定结果</span>
2104 <span
2105   className={`text-[11px] font-semibold px-2 py-0.5 rounded-full ${
2106     result.isAiGenerated
2107     ? 'bg-blue-100 text-blue-700'
2108     : 'bg-green-100 text-green-700'
2109   `}>
2110 >
2111 {result.isAiGenerated ? 'AI 生成 (含水印)' : '人类创作'}
2112 </span>
2113 </div>
2114
2115 {/* 置信度进度条 */}
2116 <div>
2117 <div className="flex items-center justify-between text-[11px] text-gray-500 mb-1">
2118 <span>AI 置信度</span>
2119 <span className="font-medium text-gray-700">{confidencePct}%</span>
2120 </div>
2121 <div className="h-2 rounded-full bg-gray-200 overflow-hidden">
2122 <div
2123   className={`h-full rounded-full transition-all duration-500 ${
2124     result.isAiGenerated ? 'bg-blue-500' : 'bg-green-500'
2125   `}>
2126   style={{ width: `${Math.max(confidencePct, 2)}%` }}
2127 </div>
2128 </div>
2129 </div>
2130
2131 <div className="grid grid-cols-2 gap-2 text-[11px]">
2132 <div className="bg-white rounded border border-gray-100 px-2 py-1.5">
2133 <div className="text-gray-400">绿名单命中</div>
2134 <div className="text-gray-700 font-medium">
2135 {result.watermarkChars} 字符
2136 </div>
2137 </div>
2138 <div className="bg-white rounded border border-gray-100 px-2 py-1.5">
2139 <div className="text-gray-400">检测耗时</div>
2140 <div className="text-gray-700 font-medium">
2141 {result.latencyMs} ms
2142 </div>
2143 </div>
2144 </div>
2145 </div>
2146 )}
2147
2148 {/* 历史记录 */}
2149 <div>
2150 <h4 className="text-xs font-semibold text-gray-600 mb-1.5">
```

```

2151 检测历史记录 ({records.length})
2152 </h4>
2153 {records.length === 0 ? (
2154   <p className="text-[11px] text-gray-400">
2155     暂无记录 · 点击"检测当前文档全文并留痕"后自动生成
2156   </p>
2157   ) : (
2158     <ul className="space-y-1.5">
2159       {records.slice(-5).reverse().map((r) => (
2160         <li
2161           key={r.id}
2162           className="text-[11px] bg-gray-50 border border-gray-100 rounded px-2.5 py-1.5 flex items-center justify-between"
2163         >
2164           <span className="text-gray-600">
2165             {new Date(r.created_at).toLocaleString('zh-CN', {
2166               month: '2-digit',
2167               day: '2-digit',
2168               hour: '2-digit',
2169               minute: '2-digit',
2170             })}
2171           </span>
2172           <span className="text-gray-400">
2173             {r.model_name} · γ={r.gamma}
2174           </span>
2175         </li>
2176       )]}
2177     </ul>
2178   )}
2179 </div>
2180 </div>
2181 );
2182 }
2183
2184 # =====
2185 # FILE: frontend/src/lib/api.ts
2186 # =====
2187 import type {
2188   Document,
2189   WatermarkDetectionResult,
2190   AgentMessage,
2191 } from '@shared/types';
2192
2193 /**
2194  * 后端 API 客户端
2195  * -----
2196  * 与 FastAPI 后端对齐 (见 /backend/api/*.py):
2197  * - 响应为裸JSON (无 {success, data} 包装)
2198  * - Agent 触发: POST /api/agents/invoke
2199  * - Agent 消息: GET /api/agents/sessions/{session_id}/messages
2200  * - 水印检测: POST /api/watermark/detect

```

```
2201  */
2202
2203  /** 后端 API 基础路径 */
2204  const BASE_URL = '/api';
2205
2206  async function request<T>(path: string, options?: RequestInit): Promise<T> {
2207    const res = await fetch(`${BASE_URL}${path}`, {
2208      headers: { 'Content-Type': 'application/json' },
2209      ...options,
2210    });
2211    if (!res.ok) {
2212      throw new Error(`API ${path} 请求失败: ${res.status} ${res.statusText}`);
2213    }
2214    if (res.status === 204) {
2215      return undefined as T;
2216    }
2217    return (await res.json()) as T;
2218  }
2219
2220  // -----
2221  // 文档
2222  // -----
2223
2224  /** 后端 DocumentOut 原始响应 (snake_case) */
2225  interface BackendDocument {
2226    id: string;
2227    title: string;
2228    owner_id: string;
2229    content: string;
2230    watermark_status: number;
2231    agent_session_id?: string | null;
2232    created_at: string;
2233    updated_at: string;
2234  }
2235
2236  /** 将后端 snake_case 文档响应映射为共享 camelCase 类型 */
2237  const mapDocument = (raw: BackendDocument): Document => ({
2238    id: raw.id,
2239    title: raw.title,
2240    ownerId: raw.owner_id,
2241    content: raw.content,
2242    watermarkStatus: raw.watermark_status as Document['watermarkStatus'],
2243    agentSessionId: raw.agent_session_id,
2244    // 后端当前不返回协作者列表, 映射为空数组 (共享类型占位字段)
2245    collaborators: [],
2246    createdAt: raw.created_at,
2247    updatedAt: raw.updated_at,
2248  });
2249
2250  /** 获取文档列表 */
```

```
2251 export const listDocuments = async () => {
2252   const raw = await request<BackendDocument[]>('/documents');
2253   return raw.map(mapDocument);
2254 };
2255
2256 /** 获取单个文档 */
2257 export const getDocument = async (docId: string) => {
2258   const raw = await request<BackendDocument>('/documents/${docId}');
2259   return mapDocument(raw);
2260 };
2261
2262 /** 创建文档 */
2263 export const createDocument = async (payload: {
2264   title: string;
2265   ownerId: string;
2266   content?: string;
2267 }) => {
2268   const raw = await request<BackendDocument>('/documents', {
2269     method: 'POST',
2270     // 后端 DocumentCreate 为 snake_case: title / owner_id / content
2271     body: JSON.stringify({
2272       title: payload.title,
2273       owner_id: payload.ownerId,
2274       content: payload.content ?? '',
2275     }),
2276   });
2277   return mapDocument(raw);
2278 };
2279
2280 /** 更新文档 (标题 / 内容); operatorId 用于溯源链操作日志 */
2281 export const updateDocument = (
2282   docId: string,
2283   payload: { title?: string; content?: string; operatorId?: string }
2284 ) =>
2285   request<Document>('/documents/${docId}', {
2286     method: 'PATCH',
2287     body: JSON.stringify({
2288       ...(payload.title !== undefined && { title: payload.title }),
2289       ...(payload.content !== undefined && { content: payload.content }),
2290       ...(payload.operatorId !== undefined && {
2291         operator_id: payload.operatorId,
2292       }),
2293     }),
2294   });
2295
2296 // -----
2297 // Agent
2298 // -----
2299
2300 /** 后端 /api/agents/invoke 的响应结构 */
```

```
2301 export interface AgentInvokeResponse {
2302   accepted: boolean;
2303   session_id: string;
2304   message: string;
2305 }
2306
2307 /** 后端 /api/agents/sessions/{id}/messages 的响应结构 */
2308 export interface AgentMessagesResponse {
2309   session_id: string;
2310   messages: AgentMessage[];
2311   total: number;
2312 }
2313
2314 /**
2315  * 触发 Agent 运行 (research / writer / supervisor)
2316  * 字段与后端 AgentInvokeRequest 对齐: doc_id, agent_type, instruction
2317  */
2318 export const triggerAgent = (
2319   agentType: 'research' | 'writer' | 'supervisor',
2320   docId: string,
2321   prompt: string
2322 ) =>
2323   request<AgentInvokeResponse>('/agents/invoke', {
2324     method: 'POST',
2325     body: JSON.stringify({
2326       agent_type: agentType,
2327       doc_id: docId,
2328       instruction: prompt,
2329     }),
2330   });
2331
2332 /** 获取 Agent 会话历史消息 (用于前端回放流式思考过程) */
2333 export const getAgentMessages = (sessionId: string) =>
2334   request<AgentMessagesResponse>(
2335     `/agents/sessions/${sessionId}/messages`
2336   );
2337
2338 // -----
2339 // 系统初始化 / 溯源链
2340 // -----
2341
2342 /** /api/bootstrap 响应: 演示用户/文档与文档列表 */
2343 export interface BootstrapResponse {
2344   demo_user_id: string;
2345   demo_doc_id: string;
2346   documents: Array<{
2347     id: string;
2348     title: string;
2349     owner_id: string;
2350     content: string;
```

```
2351 watermark_status: number;
2352 created_at: string;
2353 updated_at: string;
2354 }>;
2355 }
2356
2357 /** 获取系统初始化数据 (演示用户 + 文档列表) */
2358 export const fetchBootstrap = () => request<BootstrapResponse>('/bootstrap');
2359
2360 /** 溯源链条目 (后端 OpLogOut, snake_case) */
2361 export interface ProvenanceEntry {
2362   id: string;
2363   doc_id: string;
2364   user_id: string;
2365   op_type: string;
2366   operation: Record<string, unknown>;
2367   prev_hash: string;
2368   current_hash: string;
2369   created_at: string;
2370 }
2371
2372 /** 获取文档溯源哈希链 */
2373 export const getProvenance = (docId: string) =>
2374   request<ProvenanceEntry[]>('/watermark/documents/${docId}/provenance');
2375
2376 /** 校验溯源链完整性 */
2377 export const verifyProvenance = (docId: string) =>
2378   request<{ doc_id: string; valid: boolean; checked: number }>('
2379     /watermark/documents/${docId}/provenance/verify'
2380   );
2381
2382 // -----
2383 // 水印检测
2384 // -----
2385
2386 /** 后端 /api/watermark/detect 的原始响应 (snake_case) */
2387 interface BackendDetectResponse {
2388   is_ai_generated: boolean;
2389   confidence: number;
2390   watermark_chars: number;
2391   model_name?: string | null;
2392 }
2393
2394 /**
2395  * 水印检测 (Kirchenbauer 解码)
2396  * 将后端 snake_case 响应映射为共享类型 WatermarkDetectionResult。
2397  */
2398 export const detectWatermark = async (text: string): Promise<WatermarkDetectionResult> => {
2399   const raw = await request<BackendDetectResponse>('/watermark/detect', {
2400     method: 'POST',
```



```
2401     body: JSON.stringify({ text }),
2402   });
2403
2404   // 与 shared/types.ts 的 WatermarkDetectionResult 对齐
2405   return {
2406     docId: "",
2407     watermarkChars: raw.watermark_chars,
2408     confidence: raw.confidence,
2409     isAiGenerated: raw.is_ai_generated,
2410     latencyMs: 0,
2411   };
2412 };
2413
2414 // -----
2415 // 文献检索
2416 // -----
2417
2418 /** 文献条目 (后端 LiteratureOut, snake_case) */
2419 export interface LiteratureItem {
2420   id: string;
2421   title: string;
2422   authors: string;
2423   year: number;
2424   source: string;
2425   abstract: string;
2426   keywords: string;
2427   url?: string | null;
2428 }
2429
2430 /** 检索文献 (关键词匹配标题/摘要/关键词) */
2431 export const searchLiterature = (
2432   q: string,
2433   limit = 10
2434 ): Promise<LiteratureItem[]> =>
2435   request<LiteratureItem[]>(<
2436     `/literature/search?q=${encodeURIComponent(q)}&limit=${limit}`
2437   );
2438
2439 /** 生成文献引文 (GB/T 7714 + BibTeX) */
2440 export const getCitation = (
2441   litId: string
2442 ): Promise<{ citation: string; bibtex: string }> =>
2443   request<{ citation: string; bibtex: string }>(<
2444     `/literature/${litId}/citation`
2445   );
2446
2447 // -----
2448 // 导出 / 水印记录
2449 // -----
2450
```

```
2451  /** 导出文档为 Markdown (含溯源元数据), 返回文本内容 */
2452  export const exportDocument = async (docId: string): Promise<string> => {
2453    const res = await fetch(`${BASE_URL}/documents/${docId}/export`);
2454    if (!res.ok) {
2455      throw new Error('导出失败: ${res.status} ${res.statusText}');
2456    }
2457    return await res.text();
2458  };
2459
2460  /** 文档级水印检测: 检测全文并持久化 WatermarkRecord + 溯源链日志 */
2461  export const detectDocumentWatermark = async (
2462    docId: string
2463  ): Promise<WatermarkDetectionResult> => {
2464    const raw = await request<BackendDetectResponse>(<
2465      '/watermark/documents/${docId}/detect',
2466      { method: 'POST' }
2467    );
2468    // 与 detectWatermark 相同: 后端 DetectResponse 为 snake_case, 需映射
2469    return {
2470      docId,
2471      watermarkChars: raw.watermark_chars,
2472      confidence: raw.confidence,
2473      isAiGenerated: raw.is_ai_generated,
2474      latencyMs: 0,
2475    };
2476  };
2477
2478  /** 水印记录条目 (后端 WatermarkRecord, snake_case) */
2479  export interface WatermarkRecordItem {
2480    id: string;
2481    model_name: string;
2482    gamma: number;
2483    delta: number;
2484    created_at: string;
2485  }
2486
2487  /** 获取文档的水印检测历史记录 */
2488  export const getWatermarkRecords = (
2489    docId: string
2490  ): Promise<{ doc_id: string; records: WatermarkRecordItem[] }> =>
2491    request<{ doc_id: string; records: WatermarkRecordItem[] }>(<
2492      '/watermark/documents/${docId}/records'
2493    );
2494  # =====
2495  # FILE: frontend/src/lib/collab.ts
2496  # =====
2497  import * as Y from 'yjs';
2498  import { WebSocketProvider } from 'y-websocket';
2499  import type { Editor } from '@tiptap/react';
2500
```

```
2501  /**
2502  * 协同会话单例缓存
2503  * =====
2504  * Y.Doc 与 WebSocketProvider 按文档 ID 做模块级缓存, 由整个页面共享。
2505  *
2506  * 为什么需要单例:
2507  * 1. React StrictMode 在开发环境会 double-mount 组件 (挂载→卸载→再挂载),
2508  * 若 provider 在组件内创建/销毁, 会产生重复 WebSocket 连接;
2509  * 2. 多个组件 (编辑器 / Agent 面板 / 水印面板) 共享同一 Y.Doc 状态,
2510  * 各自创建 provider 会导致 awareness 回环 (服务端把 A 的广播转发给
2511  * B, B 应用后重新广播, 时钟无限递增, 消息风暴).
2512  *
2513  * 生命周期:
2514  * - getCollabSession(docId): 首次调用创建 Y.Doc + provider, 之后复用;
2515  * - disposeCollabSession(docId): 切换文档时显式销毁旧会话 (由 App 调用);
2516  * - 页面关闭时浏览器自动释放 (无需手动清理).
2517  */
2518
2519 export interface CollabSession {
2520   ydoc: Y.Doc;
2521   provider: WebSocketProvider;
2522   /** Tiptap 编辑器实例 (由 CollaborativeEditor 注册, Agent 面板经它插入内容) */
2523   editor: Editor | null;
2524   /** 文档初始内容是否已加载 (防 StrictMode double-mount 重复 setContent) */
2525   loaded: boolean;
2526 }
2527
2528 const sessions = new Map<string, CollabSession>();
2529
2530 /** 构建 WebSocket 地址: 开发环境经 Vite 代理转发到 FastAPI /ws */
2531 export function getWsUrl(): string {
2532   return `${window.location.protocol} === 'https:' ? 'wss' : 'ws'}://${window.location.host}/ws`;
2533 }
2534
2535 /** 获取 (或创建) 文档的协同会话: 同一文档全页面共享同一 Y.Doc 与连接 */
2536 export function getCollabSession(docId: string): CollabSession {
2537   let session = sessions.get(docId);
2538   if (!session) {
2539     const ydoc = new Y.Doc();
2540     const provider = new WebSocketProvider(getWsUrl(), docId, ydoc, {
2541       connect: true,
2542       // 二进制协议 (匹配后端 y-websocket 实现)
2543       protocols: ['yjs'],
2544     });
2545     session = { ydoc, provider, editor: null, loaded: false };
2546     sessions.set(docId, session);
2547   }
2548   return session;
2549 }
2550
```

```
2551  /** 销毁文档会话 (关闭 WebSocket 连接): 切换文档时调用, 防止连接泄漏 */
2552  export function disposeCollabSession(docId: string): void {
2553    const session = sessions.get(docId);
2554    if (session) {
2555      session.provider.destroy();
2556      sessions.delete(docId);
2557    }
2558  }
2559
2560  // 开发环境 HMR: 模块热替换时销毁全部会话, 防止旧连接泄漏与双 provider 回环
2561  if (import.meta.hot) {
2562    import.meta.hot.dispose(() => {
2563      for (const session of sessions.values()) {
2564        session.provider.destroy();
2565      }
2566      sessions.clear();
2567    });
2568  }
2569
2570  // 调试钩子: 仅开发环境暴露, 供浏览器控制台/CSP 检查协同会话状态
2571  if (import.meta.env.DEV) {
2572    (window as unknown as Record<string, unknown>).__collab = {
2573      getSession: getCollabSession,
2574      dispose: disposeCollabSession,
2575      sessionCount: () => sessions.size,
2576      /** 列出全部会话及 Yjs fragment 结构 (只读诊断) */
2577      inspectAll: () =>
2578        Array.from(sessions.entries()).map(([docId, s]) => {
2579          const frag = s.ydoc.getXmlFragment('default');
2580          const walk = (node: Y.XmlElement | Y.XmlText | Y.XmlFragment | Y.XmlHook): string => {
2581            if (node instanceof Y.XmlText) {
2582              return `text(${node.length})`;
2583            }
2584            if (node instanceof Y.XmlElement) {
2585              return `el(${node.nodeName})[${node
2586                .toArray()
2587                .map((c) => walk(c))
2588                .join(',')}]`;
2589            }
2590            if (node instanceof Y.XmlHook) {
2591              return 'hook';
2592            }
2593            return node.toArray().map((c) => walk(c)).join(',');
2594          };
2595          return {
2596            docId,
2597            connState: s.provider.ws?.readyState ?? -1,
2598            fragLen: frag.length,
2599            structure: walk(frag),
2600            plainText: frag.toString().slice(0, 120),
```

```
2601   };
2602   },
2603   };
2604   }
2605
2606   # =====
2607   # FILE: frontend\src\lib\yjs.ts
2608   # =====
2609   import * as Y from 'yjs';
2610   import type { Editor } from '@tiptap/react';
2611   import type { Document } from '@shared/types';
2612
2613   /**
2614    * Yjs 协同工具库
2615    * -----
2616    * 提供 Y.Doc 创建、文档管理与作者元数据辅助函数。
2617    *
2618    * 作者标记约定:
2619    * 文本属性 `author` = 'ai' -> 蓝色背景 (AI 生成)
2620    * 文本属性 `author` = 'human' -> 白色背景 (人类输入)
2621    */
2622
2623   export const AUTHOR_AI = 'ai';
2624   export const AUTHOR_HUMAN = 'human';
2625
2626   /** 默认 Yjs 根片段名称 (Tiptap Collaboration 扩展默认值) */
2627   export const DEFAULT_Y_FRAGMENT = 'default';
2628
2629   /** 共享文档的 state 字段: 标题 */
2630   export const META_TITLE = 'title';
2631   /** 共享文档的 state 字段: 创建者 */
2632   export const META_CREATOR = 'creator';
2633
2634   /**
2635    * 创建 (或复用) 一个 Y.Doc 实例。
2636    *
2637    * @param docId 文档 ID (后端房间 ID)
2638    * @returns Y.Doc 实例
2639    */
2640   export function createYDoc(docId: string): Y.Doc {
2641     const doc = new Y.Doc();
2642     // 文档元数据 (存储在共享 state 中, 所有协作者可见)
2643     doc.getMap('meta').set(META_TITLE, `科研文档 - ${docId}`);
2644     doc.getMap('meta').set(META_CREATOR, 'unknown');
2645     return doc;
2646   }
2647
2648   /**
2649    * 将后端返回的 Document 模型转换为 Yjs update format。
2650    */
```

```
2651 * NOTE: 这是 Stub 函数。
2652 * 真实场景中, 后端在打开文档时应返回完整 Yjs update (由 Ypy 生成),
2653 * 客户端通过 `Y.applyUpdate(doc, uint8array)` 恢复 CRDT 状态。
2654 *
2655 * TODO: 接入真实后端 Ypy 序列化状态。
2656 */
2657 export function hydrateFromBackend(
2658   doc: Y.Doc,
2659   documentModel: Document | null
2660 ): void {
2661   if (!documentModel?.yjsState) {
2662     return;
2663   }
2664   // TODO: 解析 documentModel.yjsState (base64 编码的 Yjs Update)
2665   // const binary = atob(documentModel.yjsState);
2666   // const update = Uint8Array.from(binary, (c) => c.charCodeAt(0));
2667   // Y.applyUpdate(doc, update);
2668 }
2669
2670 /**
2671 * 读取共享文档中所有已标注的作者类型集合。
2672 * 用于 UI 右上角展示"当前文档包含 AI/人类内容比例"。
2673 *
2674 * @param ydoc Yjs 文档
2675 * @returns 作者类型集合
2676 */
2677 export function collectAuthors(ydoc: Y.Doc): Set<string> {
2678   const authors = new Set<string>();
2679   const fragment = ydoc.getXmlFragment(DEFAULT_Y_FRAGMENT);
2680
2681   const walk = (node: Y.XmlElement | Y.XmlText | Y.XmlFragment): void => {
2682     if (node instanceof Y.XmlText) {
2683       // 读取文本 delta 中携带的 author 属性
2684       const delta = node.toDelta() as Array<{
2685         insert: string;
2686         attributes?: { author?: unknown };
2687       }>;
2688       for (const item of delta) {
2689         const author = getAuthorFromDelta(item);
2690         if (author) {
2691           authors.add(author);
2692         }
2693       }
2694     } else if (node instanceof Y.XmlElement) {
2695       node.forEach(walk);
2696     } else if (node instanceof Y.XmlFragment) {
2697       node.forEach(walk);
2698     }
2699   };
2700 }
```

```
2701     walk(fragment);
2702     return authors;
2703 }
2704
2705 /**
2706  * 获取当前 Yjs update (用于定期持久化/审计)。
2707  *
2708  * @returns Uint8Array Yjs 更新包
2709  */
2710 export function exportSnapshot(ydoc: Y.Doc): Uint8Array {
2711     return Y.encodeStateAsUpdate(ydoc);
2712 }
2713
2714 /**
2715  * 读取 Yjs 文本 delta 项中的作者标记, 兼容两种存储格式:
2716  * - 旧格式: 扁平字符串 { author: 'ai' }
2717  * - mark 格式: 属性对象 { author: { author: 'ai' } } (Tiptap mark 映射)
2718  */
2719 export function getAuthorFromDelta(
2720     item: { attributes?: { author?: unknown } }
2721 ): string | undefined {
2722     const raw = item.attributes?.author;
2723     if (typeof raw === 'string') return raw;
2724     if (raw && typeof raw === 'object') {
2725         const v = (raw as { author?: unknown }).author;
2726         if (typeof v === 'string') return v;
2727     }
2728     return undefined;
2729 }
2730
2731 /**
2732  * 以 AI 作者身份向文档末尾插入一段文本 (蓝色高亮演示)。
2733  * 通过 Y.XmlText 的 author 属性标记, 与 AuthorHighlight 扩展联动。
2734  *
2735  * NOTE: 仅适用于无 Tiptap 环境 (Node 单测)。浏览器中直接操作 fragment
2736  * 会被 ySyncPlugin 的 synchronize 回写规范化 (插入的 XmlText 内容被丢弃),
2737  * 请改用 appendAiText。
2738  */
2739 export function insertAiText(ydoc: Y.Doc, text: string): void {
2740     if (!text.trim()) return;
2741     const fragment = ydoc.getXmlFragment(DEFAULT_Y_FRAGMENT);
2742     ydoc.transact(() => {
2743         const paragraph = new Y.XmlElement('paragraph');
2744         const ytext = new Y.XmlText();
2745         // mark 属性对象格式: y-prosemirror 读到的 mark attrs 是 { author: 'ai' }
2746         ytext.insert(0, text, { author: { author: AUTHOR_AI } });
2747         paragraph.insert(0, [ytext]);
2748         fragment.insert(fragment.length, [paragraph]);
2749     });
2750 }
```

```
2751
2752 /**
2753  * 以指定作者身份向文档末尾追加一段文本 (浏览器主路径)。
2754  *
2755  * 为什么走编辑器通道:
2756  * ySyncPlugin 会对 fragment 的外部变更执行 synchronize (从 fragment 全量
2757  * 重建 PM 文档并回写); 直接操作 fragment 插入的 XmlText 内容会被该回写
2758  * 丢弃, 而未经 PM mark 注册的 Yjs 属性操作 (setAttribute/format) 也会
2759  * 因属性映射失败触发同样问题。因此:
2760  * 1. 用 editor.commands.insertContentAt 经 PM transaction 插入段落,
2761  * 文本携带 author mark (mark 名即 Yjs 属性名, y-prosemirror 在
2762  * PM 写回事务内自动映射为 Yjs 属性, 不会触发外部 synchronize);
2763  * 2. 不单独调用 Yjs format/setAttribute (见上述丢失机理)。
2764  *
2765  * @returns 是否已插入
2766  */
2767 export function appendText(
2768   editor: Editor | null,
2769   ydoc: Y.Doc,
2770   text: string,
2771   author: string = AUTHOR_HUMAN
2772 ): boolean {
2773   if (!text.trim()) return false;
2774   if (!editor || !editor.isEditable) {
2775     // 编辑器未就绪时退化为直接 fragment 操作 (仅保证 Yjs 数据层)
2776     const fragment = ydoc.getXmlFragment(DEFAULT_Y_FRAGMENT);
2777     ydoc.transact(() => {
2778       const paragraph = new Y.XmlElement('paragraph');
2779       const ytext = new Y.XmlText();
2780       ytext.insert(0, text, { author: { author } });
2781       paragraph.insert(0, [ytext]);
2782       fragment.insert(fragment.length, [paragraph]);
2783     });
2784     return true;
2785   }
2786
2787   editor.commands.insertContentAt(editor.state.doc.content.size, {
2788     type: 'paragraph',
2789     content: [
2790       {
2791         type: 'text',
2792         text,
2793         // mark 类型用 mark 名 'author' (与 CollaborativeEditor 注册的
2794         // AuthorMark 对应), attrs.author 才是作者值; 误用 'ai' 会导致
2795         // "There is no mark type ai in this schema" 插入被丢弃。
2796         marks: [{ type: 'author', attrs: { author } }],
2797       },
2798     ],
2799   });
2800   return true;
```



```
2801   }
2802
2803   /**
2804    * 以 AI 作者身份向文档末尾追加一段文本 (蓝色高亮)。
2805    * 供 Agent 面板在 Writer 产出后调用。
2806    */
2807   export function appendAiText(
2808     editor: Editor | null,
2809     ydoc: Y.Doc,
2810     text: string
2811   ): boolean {
2812     return appendText(editor, ydoc, text, AUTHOR_AI);
2813   }
2814
2815   /**
2816    * 读取文档纯文本 (供水印检测 / 持久化 / 导出使用)。
2817    * XmlFragment.toString() 会输出 XML 标记, 这里按 paragraph 分段提取,
2818    * 段落间以换行分隔, 空段落忽略, 段内换行保留。
2819    */
2820   export function getPlainText(ydoc: Y.Doc): string {
2821     const fragment = ydoc.getXmlFragment(DEFAULT_Y_FRAGMENT);
2822     const paragraphs: string[] = [];
2823     const current: string[] = [];
2824     const walk = (node: Y.XmlElement | Y.XmlText | Y.XmlFragment): void => {
2825       if (node instanceof Y.XmlText) {
2826         current.push(node.toString());
2827       } else if (node instanceof Y.XmlElement) {
2828         if (node.nodeName === 'paragraph') {
2829           const text = current.join("");
2830           if (text.trim()) paragraphs.push(text);
2831           current.length = 0;
2832         }
2833         node.forEach(walk);
2834       } else {
2835         node.forEach(walk);
2836       }
2837     };
2838     walk(fragment);
2839     const tail = current.join("");
2840     if (tail.trim()) paragraphs.push(tail);
2841     return paragraphs.join("\n");
2842   }
2843   # =====
2844   # FILE: frontend/src/main.tsx
2845   # =====
2846   import React from 'react';
2847   import ReactDOM from 'react-dom/client';
2848   import App from './App';
2849   import './index.css';
2850
```

```
2851 ReactDOM.createRoot(document.getElementById('root')).render(  
2852   <React.StrictMode>  
2853     <App />  
2854   </React.StrictMode>  
2855 );  
2856 # =====  
2857 # FILE: frontend\src\vite-env.d.ts  
2858 # =====  
2859 /// <reference types="vite/client" />  
2860  
2861 # =====  
2862 # FILE: shared\types.ts  
2863 # =====  
2864 /**  
2865  * 前后端共享类型定义  
2866  * 面向科研诚信的多Agent实时协同与版权溯源系统  
2867  */  
2868  
2869 // ===== 用户 =====  
2870  
2871 export interface User {  
2872   id: string;  
2873   username: string;  
2874   email: string;  
2875   displayName: string;  
2876   avatarUrl?: string;  
2877   role: UserRole;  
2878   createdAt: string; // ISO 8601  
2879 }  
2880  
2881 export enum UserRole {  
2882   ADMIN = 'admin',  
2883   RESEARCHER = 'researcher',  
2884   COLLABORATOR = 'collaborator',  
2885   AI_AGENT = 'ai_agent', // AI Agent 在系统中也以特殊用户身份参与  
2886 }  
2887  
2888 // ===== 文档 =====  
2889  
2890 export interface Document {  
2891   id: string;  
2892   title: string;  
2893   ownerId: string;  
2894   /** 协同状态快照 (Yjs Update 二进制序列化为 base64) */  
2895   yjsState?: string | null;  
2896   /** 文档当前内容 (明文快照, 便于快速预览) */  
2897   content: string;  
2898   /** 水印标识: 0-未检测 1-存在AI水印 2-纯人类创作 */  
2899   watermarkStatus: WatermarkStatus;  
2900   /** 关联的 Agent 会话 ID */
```

```
2901     agentSessionId?: string | null;
2902     collaborators: string[]; // userId 列表
2903     createdAt: string;
2904     updatedAt: string;
2905 }
2906
2907 export enum WatermarkStatus {
2908     UNKNOWN = 0,
2909     AI_WATERMARKED = 1,
2910     HUMAN = 2,
2911 }
2912
2913 // ===== 操作日志 (版权溯源) =====
2914
2915 /**
2916  * 操作日志形成哈希链:
2917  * prev_hash -> current_hash(sha256(prev_hash + operation_json + timestamp))
2918  * 任一历史操作被篡改, 整条链即断裂, 可被服务端校验。
2919  */
2920 export interface OpLog {
2921     id: string;
2922     docId: string;
2923     userId: string;
2924     /** 操作类型: insert | delete | replace | ai_generate | watermark_checked */
2925     opType: string;
2926     /** 操作内容 (JSONB) */
2927     operation: Record<string, unknown>;
2928     prevHash: string;
2929     currentHash: string;
2930     createdAt: string;
2931 }
2932
2933 // ===== Agent 消息 =====
2934
2935 export enum AgentType {
2936     RESEARCH = 'research',
2937     WRITER = 'writer',
2938     SUPERVISOR = 'supervisor',
2939 }
2940
2941 export enum AgentStatus {
2942     IDLE = 'idle',
2943     RUNNING = 'running',
2944     WAITING_HUMAN = 'waiting_human',
2945     COMPLETED = 'completed',
2946     ERROR = 'error',
2947 }
2948
2949 export interface AgentState {
2950     type: AgentType;
```

```
2951     status: AgentStatus;
2952     /** 目标文档 ID */
2953     docId?: string;
2954     /** 当前思考步骤描述 */
2955     currentStep?: string;
2956     startedAt?: string;
2957     finishedAt?: string;
2958   }
2959
2960   /** Agent 实时流式消息 (通过 WebSocket / SSE 推送) */
2961   export interface AgentMessage {
2962     id: string;
2963     sessionId: string;
2964     agentType: AgentType;
2965     role: 'agent' | 'human' | 'system';
2966     content: string;
2967     /** 思考过程标签: planning | search | reasoning | writing | review | done */
2968     phase: string;
2969     createdAt: string;
2970   }
2971
2972   // ===== 水印检测结果 =====
2973
2974   export interface WatermarkDetectionResult {
2975     docId: string;
2976     /** 检测到的水印字符数 */
2977     watermarkChars: number;
2978     /** 置信度 0~1 */
2979     confidence: number;
2980     /** 是否判定为 AI 生成 */
2981     isAiGenerated: boolean;
2982     /** 检测耗时 ms */
2983     latencyMs: number;
2984   }
2985
2986   // ===== API 通用响应 =====
2987
2988   export interface ApiResponse<T = unknown> {
2989     success: boolean;
2990     data?: T;
2991     error?: string;
2992   }
2993
2994   /** WebSocket 消息协议 */
2995   export type WsMessage =
2996     | { type: 'connected'; clientId: string; docId: string }
2997     | { type: 'agent_update'; payload: AgentMessage }
2998     | { type: 'agent_state'; payload: AgentState }
2999     | { type: 'sync_ready'; docId: string; clientCount: number }
3000     | { type: 'error'; message: string };
```